

Parts User Guide

Table of Contents

Parts User Guide	1
Introduction	5
Our Philosophy	5
How SCons Works	5
So what is Parts?	5
Setup	6
Parts Concepts	7
Parts and Sub Parts	7
Versioning	7
Dependency mapping	7
Tool Chains	7
Configurations	7
Tools	8
Pieces	8
Mappers	8
User API	9
Component Handling	9
The Part concept	9
Component API calls	9
Version handing	13
Version Object	13
Version Range object	16
Version information	18
Version information API calls	18

Dependence mapping	20
Matching issues.....	20
Dependency mapping API calls	20
Configurations	25
Configuration API	25
Platform Handling	27
Environment Variables.....	27
Adding new platforms.....	27
Tool Chains	29
Tool chain API calls.....	29
Node helpers	30
Node helper API calls	30
The Pattern object	33
SDKs and Exporting	35
The SDK Methods.....	35
Packaging.....	39
Install sandbox	39
The Install API.....	39
Package Groups.....	41
The Package Group API	41
The Package Group Criteria API	44
Version control system.....	46
Vcs API.....	46
Testing	48
Python script runner	51
Python script runner API	51
Documentation building.....	52

Configuring Parts	52
Command line vs. Variable	52
Variable Usage	53
Order of importance	53
Configuration File	53
Command Line Usage	55
Output Options:	55
Build Options:	59
VCS Options:	62
General Options:	65
Variables	70
Packaging	70
Install	70
Sdk	72
Building	75
HOST_PLATFORM	75
Unit testing	76
Part Variables	77
Logging	79
VCS	79
Miscellaneous	81
Extending Parts via Parts-site	82
Locations	82
Defining a piece	82
Defining your own Tool Chain	82
Defining your own Configuration	87
Layout of configurations	87

Defining a configuration.....	87
Define a file name	87
Define settings for a tool.....	90
Simple example of Configuration	91
Adding Tools with Parts.....	92
Basic requirements	92
ToolSetting and ToolInfo.....	93
Example of a Tool Definition:	97

Introduction

SCons (from *Software Construction*) is a cross-platform, open-source, make-replacement, build system written in Python.

While *makefiles* have a specialized, sometimes arcane, syntax that is interpreted by the make utility, the configuration files for SCons are Python code.

The use of Python enables a clean and simple way to define scripts without a need to know Python in detail. It also enables an easy way to extend SCons and use Python facilities to debug and solve issues that are difficult to solve in classic Make or IDE project systems.

Our Philosophy

- Any developer can build the product: A product should always be build-able by a developer, not just on a special box or by special people with special knowledge of what special items must be set up to make the build work.
- Extend SCons, do not wrapper it: Instead of making a tool that hides the use of SCons, we use the ability of SCons to extend logic and functionality naturally. This means we use SCons as is, but get some extras for free. It also means the ideal build functionality of SCons should work as documented, and our new logic should work on top of this. However, nothing is perfect and there are a few rough edges.

How SCons Works

At a high level, SCons is just a running Python script. The way SCons is defined, you can write a powerful build script in SCons without being a Python expert or even knowing you are using Python. The general flow of a SCons build is as follows:

1. Invoke SCons.
2. Process any command-line arguments.
3. Look for a SConstruct file (equivalent to a makefile for Make users).
4. Read this SConstruct file (referred to as the “read phase”)
5. Process the resulting actions described in the SConstruct file until it is done (referred to as the “execution/build phase”).

So what is Parts?

One of the core issues with SCons is its inability to scale to large projects. Much like Make systems, SCons’ lack of structure idioms prevents different groups in large development projects from easily sharing code in a plug-and-play fashion.

To help address this lack of structure, we defined a concept called a *Part*. A Part is much like a *component*. In fact the two are generally interchangeable; however, we used a different term that does not collide with any preconceived notions of what a component might be.

The Python Part function in the SConstruct file loads another file called the parts file to get information on how to build a given component. It is much like the SCons SConscript() function. The difference is that the Part function sets up a common structure to prevent redefinition of common concept idioms in incompatible ways. This standardizing and extension of these build idioms is the core idea of what a Part is and what the Part function does. The end result is an easy way to build a product with hundreds or thousands of components, while allowing you to easily plug-and-play each component into one or more products or personal development builds.

The natural design of SCons lets you define *what* you want to happen, not *how*, thus providing a better way to define how to build code in a cross platform way. Instead of building files with a specific tool in a specific order in a specific shell environment, you can tell SCons to build a result from specific sources without worrying about what tools to use, the order of the sources to be built, the exact location of the tools on the given system, the shell used, or even the platform. SCons also lets you try different tools chains at a higher level and create portable scripts.

Notice that we refer to the *Parts* extension to SCons which introduces a *Part* function that defines a component-like entity called a *Part*. And one or more of those entities are collectively called *Parts*! Further, the script that describes how to define one of those entities is called a *Parts* file!! This document will try to provide enough contexts to avoid confusion of definition.

Setup

To setup SCons and Parts on your system you will need to have an installed version of Python. For many POSIX based systems, there is a predefined python package to install, or the system may already have it installed. For Windows, you will have to install Python by downloading a package from the internet, as Windows does not ship with it. We recommend the python package from ActiveState. You will want Python version 2.6 or better to use Parts. As of the time of this writing Python 3.0 is not supported.

You will want to download the latest stable version of SCons from www.scons.org. Currently this is version 2.1.0. Follow the instructions for installing SCons on the website. If you are a windows user, they have a nice Windows installer

To get Parts you can download one package from parts.tigris.org or check out the code with SVN. There should also be a nice installer for windows provided. Just make sure you uninstall any previous versions first. If you checkout or download a compressed file, you will need to do these steps to install

1. unpack the file into a directory (only needed if you did not checkout a tagged or trunk version)
2. open a command prompt and go to the directory where the code was checked out
3. elevate permission to allow for a safe install
4. run the command "python setup.py install"
5. to verify install, run the command "parts". (This will print out the version installed)

Parts Concepts

Parts adds a number of new concepts to SCons and tries to enhance some existing ones. This section describes these concepts

Parts and Sub Parts

The most noticeable concept that Parts adds to SCons is the structured notion of a component, which is called a Part. This idea is basically defining a file that is provided a preconfigured build environment and an extended API. This allows different component owners or library writers to provide a simple cross-platform build file that can be added into any other project that uses SCons and Parts without having to resort to massive amounts of tweaking to get it to build or have the component be used by other components. To help with this each Part has a number of traits. These traits allow for a number of other concepts to be defined and used. These traits include (not complete):

- alias - a unique identifier for every Part defined
- name - a constant identifier for a given Part even if that Part is defined more than once with different configuration values
- version - a value to say how new or old the Part is

Versioning

Most systems have some notion of a version. However it is often treated as a random string value that cannot be used for testing if something is greater or less than something else. Parts adds the notion of a formal Version object that is currently limited to three numbers separated by a <dot> "." In addition to a version object there is also the notion of a Version Range which allow for testing if a version is within a range. This ability allows for easy requirement testing and version matching between different Parts.

Dependency mapping

Parts adds the ability to map dependencies between different parts. This is much like Mapping dependencies between projects in Visual Studio IDE project system, or other similar systems. Parts adds the ability to implicitly map paths and other values needed to build dependent components. This allows for a more agile build system that will help prevent and even find badly mapped values. This increases productivity and confidence in the build, especially for large projects.

Tool Chains

Tool chains are sets of tools that are used to construct an environment. These sets allow the use of different compilers, or adding library environments or unique builder API's needed in the build. These named sets can be defined by the user to provide the exact requirement for a given build no matter how "picky" or flexible you want them to be. These can be installed independent of the Parts install.

Configurations

These are named collections of settings to apply when building the product. These allow for control over which flags and settings are used. Common configurations are 'debug' and 'release'. Users can define their own configurations based on existing or new configuration values. These can be installed independent of the Parts install.

Tools

Tools defined by Parts have a slight enhancement to help with consistency. All Tools can select different versions of themselves. Each can be configured for cross builds or can be setup with a user script. To help with defining new tools, Parts adds some objects for general configuration and setup of a given tool's environment. Any existing tool in SCons can be used as is, as long as there is not a Parts override for it.

Pieces

To help make SCons easier to add-on to, Parts adds a notion of 'pieces' that are python files that are loaded at startup and can add about any set of functionality to SCons or Parts that can be imaged.

Mappers

Mappers are a more formalized object that enhances the SCons subst() feature. This object allows Parts to map data dependencies as data between different components in a way that allows the parts to be defined in the SConstruct independent of the order. Mappers have been designed to allow user defined objects to be loaded and to extend the functionality of Parts. Some useful predefined Mapper objects allow for creation of relative paths between different locations, or adding data from a part to the environment of another. (This is how Parts passes information such as include paths to be used)

User API

Component Handling

The ability to handle the notion of a component is defined by creating a Part. The primary benefit of defining a Part vs. a recursive build file, as you might in Make, is the ability to better reuse the Part in a different products or projects without having to make a new build file or having to do multiple build passes in case of cross building.

The Part concept

A Part provides a more organized way to group pieces of a product into smaller components. A Part itself may contain a set of Parts, i.e. sub-Parts, itself. A Part itself is an object that contains information about the name, version and other values that define the component. Each Part can contain one or more sections that define dependencies and what is being built. These basic attributes allow for an organized and systematic way to allow small to large sets of different Parts to work and build together without having to set low level build rules for each component. Instead these can be controlled at a more macro level to allow components to be more easily shared together in a cross platform way. This allows developers to work more on the code, and worry less on how it will be built on different platforms or tool chains.

The Part file

Each Part is defined by file that normally ends with .parts or .part by convention. Before SCons loads a Part file a number of variables and values are setup in a cloned environment based on the current state of the default SCons environment and values passed in to the Part() API. These value can be imported via a SCons Import('*') call at the start of the Part file. The environment that is created for the Part is passed in as the variable “**env**”.

Sub-Parts

Calling Part in another Part file will define a Sub-Part. There is no limit, except any technical python or system memory limit, on the depth or the number of Sub-Parts that can be defined. It is however recommended that this ability is used sparingly is at all. The larger the number of sub-parts that are created the harder it is to share with other users. Getting what is needed by the dependent will become more complex, as well the about of bloat and unused sub-part can increase. This leads to the need for extra disk space and processing time to deal with defined, but unused targets.

Component API calls

```
Part(parts_file=None,mode=[], vcs_type=None,default=False, append={}, prepend={},  
create_sdk=True, package_group=None, alias=None, name=None, **kw)
```

Define a Part of Sub-Part to the build system.

Arguments:

- `part_file` – the file to load that defines the Part. By default this is relative from the defining file. If `vcs_type` is used it is relative to the value of the `CHECK_OUT_DIR` variable.
- `mode` – list of value that the Part file can use to control custom behavior
- `vcs_type` – pass the vcs object that defines how to checkout or copy data from some version control source such as CVS or SVN.
- `default` – will have this Part build by default if no target is given.
- `append` – is a dictionary of key values where each existing key will have its value appended to the value of any variable defined in the default environment, during construction. If the key is not defined, the key will be added as is. It is recommended to use the `ConfigValue()` API to define value for this argument.
- `prepend` – like `append` argument but will try to prepend the value list instead.
- `create_sdk --` tells Parts to create a SDK for this Part, or to skip it. By default the creation of a SDK will be attempted.
- `package_group` – Tell Parts what package group to map installed item to for the Part. This is used by the Packaging API and builder to help create install packages.
- `**kw` – this allows random values to be added or replace variables defined in the SCons environment created by Parts.

Access as:

- Function in SConstruct file
- Method of SCons Environment
- Function in Parts file

Example:

Creates a Part from within the SConstruct

```
Part('filesystem.parts')
```

Defines a Part to be checked out of SVN

```
Part('file_finder.parts', vcs_type=VcsSvn(
server="https://my_subversionserver",
repository='static/file_finder_1.0.0') )
```

Define a Part that will add a special CPPDEFINE value

```
Part(parts_file='filesystem.parts',
append={ 'CPPDEFINE' : [ 'FOOBAR', 'DEBUG' ] })
```

PartName(id=None)

This method defines the name of a given Part. If a name is not given then the Parts will use a unique ID generated by the build system. The advantage of defining a name is that a constant value will be used between different SConstructs files. This method if given a value will set the name to the given string value, if given nothing it will return the full name of the Part. For Sub-Part the value entered will be prepend with the parent Part name and separated with a "." dot character.

Arguments:

- id – if not equal to None, will set the name of the Part to the given string value

Access as:

- Function in Parts file

Examples:

Set the Part name to "foo"

```
PartName("foo")
```

Get the full name of the Part

```
PartName()
```

PartShortName()

Returns the short name of this Part. This is currently defined as the value originally pasted to the PartName function, or if this was not called it would fall back on to the value generated by the system.

Arguments:

None

Access as:

- Function in Parts file

Examples:

Get the Short Name of the Part

```
Name=PartShortName()
```

Show how ShortName would map in case of a Sub-Part with a parent Part named “root”

```
PartName("child")  
  
print "Full name:",env.PartName()  
  
print "Short name:",env.PartShortName()
```

Would print this to the console

```
Full name: root.child  
Short name: child
```

PartVersion(ver=None)

This method defines the version of a given Part. It will take a string or Version object to define a version. If not given a value, it will return, as a Version object, the version of the Part. The version is shared between all Parts and Sub-Parts. If no version is defined the version is automatically set to “0.0.0”.

Arguments:

- ver – if not equal to None, will set the version of the Part to the given string value in the form of “A.B.C”, or will take a Version object.

Access as:

- Function in Parts file

Examples:

Set the Part version of the Part to 1.23.8 using a string

```
PartVersion("1.28.3")
```

Set the Part version of the Part to 1.23.8 using a Version object

```
PartVersion(Version("1.28.3"))
```

Get the full version of the Part

```
ver=PartVersion()
```

PartShortVersion()

The short version is defined as the A.B portion of a version number. This value is returned as a version object with the C value unset. If the version is not set the version returned is 0.0.

Access as:

- Function in Parts file

Argument:

None

Examples:

Get the short version of the Part

```
ver=PartShortVersion()
```

Version handling

Being able to manage different versions of Parts or tools, etc., requires a standard way to represent a version number. To support this, the concept of a version object is defined by Parts. While a version number definition differs from different project and groups, the core of what a version number consists of is often the same. Because of this, Parts tries to address what is common in most version numbers which is a set of values separated by “.”dots. The primary problem which deals with version numbers is how to compare them. Worst case a version is equal or not equal to a different version. The common case is that a newer version has an incremental, sortable value that is greater than the previous value. The version object in Parts provides the basic comparison operators to allow for easy handling of a version number. The concept of a version range is also provided as a natural way to handle varying version matching requirements. However it is not perfect and cases can be defined that are difficult to sort. Parts defines two different concepts of a version. One is a version value, such as 1.2 or 1.4a.5.Beta. The other is a version range which defines a range in which a provide version can exist in.

Version Object

This object provides the basic abilities needed to handle the version concept. Comparison operators allow for the ability to test if a version is greater or less or equal to some other version value. A minus operator is provided to create ranges. String values that are not numbers are handled in two different ways. The first way is to see if it is a special value, such as **Beta** or **Alpha**. If it is one of these values it get assigned a special number (a large negative number) to allow it to be sorted. If it is not one of these special values, the number is sorted by the value if it was a string being compared to a string. It is a little more complex than this as the values such as 3a get split into 3 and “a”. The goal is that values such as 1.2.3a are greater than a value of 1.2.3, but 1.2.3a is less than 1.2.4. The current sets of special string are based on common value that normally used today. Some values are equal to other values, for example release and gold are viewed as equal. Current values are (to be added to, values on same line are viewed as equal, will change based on feedback):

- Dev
- Alpha
- Beta, ctp

- rc, pc
- rtm, release, final, gold

It should be noted that these special values may have different views depending on where you work or what you do. It suggested that values that have the same value might be viewed as different stages an item can go through. For example a 1.2.Beta build of a product might need to be tested in a certain way for it to be viewed good enough to be a 1.2.CTP (Customer Tech Preview). Since this can get complex quickly at certain organizations, it might be best to separate the version from the stage of the bit, and not combine them.

*Version(*lst)*

- *lst – this value if provided as a string in the format “A.B.C” will be parsed and separated, If provided as a version object, it will copy the value of the version object. If this is the None object the version will be set to a state of None, in which all values will be Zero, but the object will not be equal to 0.0.0, but None. The common case will be that this value is set as a number in integer form or string form, in which case it will be assumed to be the A or Major value of the version

Access as:

- Method of SCons Environment
- Object in SConstruct file
- Object in Parts file

Examples:

Create a version object with a string

```
Version("1.2.3a")
```

Create a version object with a numbers

```
Version(1,2,3,7235)
```

Create a version object with a numbers and strings

```
Version(1,2,"beta",7235)
```

Defines version object

```
env.Version("1.2.3")  
env.Version(1,2,3)  
v=env.Version('1',2,'3')  
env.Version(v)
```

Methods:

VERSION.OPERATORS <,>,<=,>==,!=

Comparison operators to control test if a version is equal, not equal, less than or greater than to another version object or string that is convertible to a Version object

EXAMPLES:

Test if a version is less than another

```
Version("1.2.3") < Version("1.2.4")
```

Test if a version is greater or equal to another as a string

```
Version(1,8,3) >= "1.3.4"
```

VERSION.OPERATORS []

Allows access to a given number value in the version

EXAMPLES:

Get the major version number

```
Version(1,2,3)[0] #-> 1
```

VERSION.OPERATORS [:] (SLICE)

The splice operator allows access to a range of numbers in the version

EXAMPLES:

Get the major and minor version numbers

```
Version(1,2,3)[0:2] #-> 1.2
```

VERSION.OPERATORS __LEN__

Return the length of the number of different numbers in the version

EXAMPLES:

Get the major version number

```
len(Version(1,2,3,4,5,6,7,8)) #-> 8
```

Version Range object

The version range object is provided to easily define a range of valid versions to match a given version object to. The version range allows noncontiguous ranges to be defined. The use of * is allowed to define number value. The – is used to define a range. The “!” operator is used to define not this version and “,” comma can be used as a separator value. This object provides an operator overload for use with the python “in” operator. It should be noted that a range is define include the first number and to exclude the last number. Mathematically the default range is **[a-b)**. This mean that **1-2** is the same as **[1-2)**, or to say that 2 is not within the range. This can cause some confusion to some people when you consider that **1-1.*** is the same as **1-2**, which both mean that max value is 1.<any value> or 1.<infinity-1> is a good value. The values of “(“ and “)” can be added to clarify any meaning, for example you can say **1-2]** which would mean the value of 2.0.< infinite 0 value being added> would be in the range, but 2.0.< infinite 0 value being added>.1 would not be. Doing this is generally not useful; however this seems to be a point of technical concern for some.

VersionRange(range=None)

- range – a string value describing the range, or another VersionRange object

Access as:

- Method of SCons Environment
- Object in SConstruct file
- Object in Parts file

Examples:

Make a version range

```
env.VersionRange("1.2.3")
env.VersionRange(env.Version("1.2000.312334"))
env.VersionRange("1.*")
env.VersionRange("1.*.3")
env.VersionRange("3.2.*")
env.VersionRange("1-2")
env.VersionRange("1-*")
env.VersionRange("1.*-2")
env.VersionRange("1-2,!1.5")
env.VersionRange("*-2")
env.VersionRange(["*-2","4-5"])
env.VersionRange(env.Version("1.0.0") - env.Version("2.0.0"))
```

Methods:

VERSIONRANGE.OPERATORS __CONTAINS__

This operator allow for simple testing of a version object is within a version range.

EXAMPLES:

Testing to see if a version is within a range

```
'6.0' in VersionRange('7.1-7.*') # false
Version('6.0') in VersionRange('7.1-7.*') # false
'7.5' in VersionRange('7.1-7.*') # true
'7.5' in VersionRange('7.1-7.*,!7.5') # false
'6.0' in VersionRange('7.1-7.*,6.*') # true
'6.0' in VersionRange('*-2') # false
'1.4.0' in VersionRange('*-2') # true
'1.9.9.9.9' in VersionRange('1-2') # true
'1.9.9.9.9' in VersionRange('1-1.*') # true
'2' in VersionRange('1-2') # true
```

Version information

Parts defines a function to provide information about which version of Parts is currently being used. This allows user to report, and control functionality of their Part file depending on what version is being used.

Version information API calls

PartVersionString()

This function returns the Part version as a formatted string.

Arguments:

None

Access as:

- Method of SCons Environment
- Function in SConstruct file
- Function in Parts file

Examples:

Make a string with the version in it

```
print PartVersionString()

# would print

Parts extension for SCons, Version 0.10.1
```

IsPartsExtensionVersionBeta()

This function returns a True if the current version of Parts is considered to be in Beta

Arguments:

None

Access as:

- Method of SCons Environment
- Function in SConstruct file
- Function in Parts file

Examples:

Tests to see if this version is Beta, if so then try a new feature

```
if IsPartsExtensionVersionBeta() and PartsExtensionVersion() >= "0.10.2":

    # use new feature

else:

    # do it the old fashion way.
```

PartsExtensionVersion()

This returns the Part version as a Version object.

Arguments:

None

Access as:

- Method of SCons Environment
- Function in SConstruct file

- Function in Parts file

Examples:

Print a warning message if the Part version is too old

```
if PartsExtensionVersion() < "0.10.0":  
    PrintWarning("Should use Parts 0.10.0 or better")
```

Prevent build if version of Parts is too old

```
if PartsExtensionVersion() < "0.10.1":  
    PrintError("Should use Parts 0.10.1 or better")
```

Dependence mapping

Mapping of dependent components is a key requirement for dealing with other components that may have changed API's or behaviors between different versions. This allows for a basic yet powerful way to share information between components. Information can be basic information that is needed to build a component, such as the libraries one needs to link with, or the path to some headers or modules needed to compile. Custom data can also be shared, allow special values such as name of a file to load, or custom flags to use in certain cases when building against component libraries. For any given mapping the user can provide certain requirements that all control over what data is imported and how it might be used and what version should be matched. Parts will try to map the best match, being the component that meets all requirements and is the highest version to map in a request version range.

Matching issues

If a match cannot be found the system will report an error and information about possible matched. If more than one version is available leading an ambiguous match the system will report an error on which components are causing a problem.

Dependency mapping API calls

DependsOn (depends)

This function takes a list of components or strings that defines the dependence requirements to build the given component. It can be call more than once, with each call adding to the dependency requirements. Information that is requested in the dependency requirements will not be available until after the function call.

Arguments:

- depends – a string or Component or list of Components/strings that defines which Part names the given component depends on. String values are made into a Component with default values.

Access as:

- Method of SCons Environment

Example:

Depends on Part with the name “base” which is passed as a string

```
env.DependsOn ("base")
```

Depends on Part with the name “base” but limited to version 2.*

```
env.DependsOn (Component('base', '2.*'))
```

The same as above but as a list and depends on any version 2.* except 2.5

```
env.DependsOn ([Component('base', '2.*', !2.5)])
```

Depends on a component “infra” where the headers dependence is passed on but the linking is kept private to this component.

```
env.DependsOn ([Component('infra', '2.*', requires=REQ.HEADERS|  
REQ.LIBS(internal=True)])]
```

Component(name, version_range='', requires=REQ. DEFAULT)*

The component object defines a combination of a Part name, version range and dependence requirements needed by the Part being defined.

Arguments:

- name – a string, which is the full Part name of the Part we want to depend on
- version_range – a string or VersionRange object that describes a pattern of best valid versions to allow mappings to.
- Requires – a set of bits defined by the REQ object, used to describe how to map libs, path defines, and flags of the depending Part to the current defining Part.

Access as:

- Method of SCons Environment
- Function in Parts file

Example:

Creates a component to map version 2.5.3 of a Part named “parser”

```
Component ('parser', VersionRange('2.5.3'))
```

The same but with everything as a string

```
Component ('parser', '2.5.3')
```

Component that requires headers information only

```
Component ('infra', '2.*', requires=REQ.HEADERS)
```

REQ.<MetaValue>, REQ.<MetaValue>(internal=None, public=None, policy=None)

The REQ object, short for requirements, defines a bunch of Meta values that describe what dependency information to map from component and how it should be mapped. To use this object, one would define one or more Meta values and have them logically ‘|’ together to define a complete set of requirements to have passed. The Meta value can be defined as a function to allow more control over how the requirement should be handled when mapping information. User values are exported via the export API.

Arguments:

All arguments map to None by default which means it will use any predefined behaviors for the given Meta value. In general this means internal=False, public=True, policy=REQPolicy.ignore for predefined Values. For custom values this means internal=False, public=False, policy=REQPolicy.warning.

- Internal – Control if this information should be passed implicitly to components that depend on current Part being defined, or is the information should only be seen by the current Parts. In future drops the default may change to make everything internal by default.
- Public – controls if the information is defined in the top level of the environment object, adding to the currently defined values, or if the information is put in a private namespace under the environment. If private the value is found under the env[‘DEPENDS’][<partname>][<keyvalue>]
- Policy – control behavior on how a missing data should be handled in cases when the information is not exported by the component. It should be of type REQPolicy, which allows one to ignore, print a verbose message of category *req*, warn, or error in case of failure.

Access as:

- Object in Part File

Predefined values:

Most of the predefines are focused on C or C++, however expect this to change as better support for different tools get added.

REQ.EXISTS

Used to just say that the component we depend on is required to be built and up to date when defining component is built. Useful for components that will dynamically load another component or if the other Part may output data files needed to control the runtime logic of the current Part.

REQ.CPPPATH

Passes the path needed to include the header files. This only passes the path, while HEADERS also passes CPPDEFINES needed to process the headers correctly.

REQ.HEADERS

Like CPPPATH but also passes the CPPDEFINES needed to correctly process the headers if any are defined.

REQ.LIBPATH

Pass the LIBPATH value to the dependent Part to help link with any libraries defined by the given Part. Paths of the Libs added to the SdkLibs, Sdk or SdkTarget functions will be used.

REQ.LIBS

Pass the LIBPATH value and the LIBS to the dependent Part to automatically link with. Libraries added to the SdkLibs, Sdk or SdkTarget functions will be used.

REQ.LINKFLAGS

Pass exported LINKFLAGS to current Part. Flags passed to the ExportLINKFLAGS will be used.

REQ.CCFLAGS

Pass exported CCFLAGS to current Part. Flags passed to the ExportCCFLAGS will be used.

REQ.CFLAGS

Pass exported CFLAGS to current Part. Flags passed to the ExportCFLAGS will be used.

REQ.CXXFLAGS

Pass exported CXXFLAGS to current Part. Flags passed to the ExportCXXFLAGS will be used.

REQ.CPPDEFINES

Pass exported CPPDEFINES to current Part. Flags passed to the Export CPPDEFINES will be used.

REQ.C_DEFAULTS

Pass exported HEADERS and LIBS to current Part.

REQ.CPP_DEFAULTS

Pass exported HEADERS and LIBS to current Part.

REQ.DEFAULT

The default is currently defined as REQ.CPP_DEFAULTS|REQ.C_DEFAULTS|REQ.INSTALLFILES|REQ.SDKLIB|REQ.SDKBIN. (expect these value to change over time)

REQ.SDK<XXX>

Pass exported values of the give SDK API call. We don't list them all here as there is a one to one mapping with the SDK API. These have a default value internal=True

REQ.SDKFILES

Pass exported values all the SDK calls. Has a default value internal=True.

REQ.INSTALL<XXX>

Pass exported values of the give SDK API call. We don't list them all here as there is a one to one mapping with the SDK API. These have a default value internal=True

REQ.INSTALLFILES

Pass exported values all the SDK calls. Has a default value internal=True.

Examples:

You want to copy the installed binary of a component to a special area to do testing.

Important!

- Dependency must be defined first!
- Part A has to be defined before this Part in the Sconstruct for this to work!

```
env.DependsOn ([Component ('A', requires=REQ.INSTALLTARGET) ] )  
  
a_bin=env.subst ("${DEPENDS.A.INSTALLTARGET} ")  
  
env.CCopy ("#TESTING/foo", a_bin)
```

I want to export some custom data for others to get.

Parts A

```
Env.ExportItem("ConfFile","foo.cfg")
```

Part B

```
env.DependsOn([Component('A',requires=REQ.CONFFILE|REQ.DEFAULT)])  
data=env.subst("${DEPENDS.A.CONFFILE}")
```

Or maybe you want to map this in a less private space.

```
env.DependsOn([Component('A',requires=REQ.CONFFILE(public=True)|  
REQ.DEFAULT)])  
data=env.subst("${CONFFILE}")
```

I need to depend only on the headers of another component

```
env.DependsOn([Component('A',requires=REQ.HEADERS)])
```

I only want the defaults

```
env.DependsOn([Component('A',requires=REQ.DEFAULTS)]) #or  
env.DependsOn([Component('A')])
```

I want to depend on A but it use is hidden in my component.

```
env.DependsOn([Component('A',requires=REQ.DEFAULTS(internal=True))])
```

Configurations

Configurations are name sets of flags and settings that can be applied to the environment. This extension to SCons by Parts allows users to better define and apply flags to the build for small to large projects. Configurations by their design can reuse base configuration settings and modify them with new values or completely replace them. This allows for chains of configurations with common base type such as debug or release to be defined. For example it is possible to make a chain such as:

debug -- shipped with Parts

<org>_debug -- settings overrides or additions required by the organization

<product>_debug -- settings overrides or additions required by the product

See section on how to define your own configurations for more details.

Configuration API

isConfigBasedOn (name)

The configuration being used can be found from the \$CONFIG value in the environment. However this does not allow the user to tell if the configuration is based on some other configuration. For example, the user will want to

know if the build is based on debug or not. To find if the configuration is equal to or based on some other known type of configuration, `isConfigBasedOn` should be called.

Arguments:

- `name` – is the name of the configuration you want to compare with.

Access as:

- Method of SCons Environment

Example:

See if the current configuration is based on or equal to the debug configuration

```
if env.isConfigBasedOn('debug') :  
    #add special defines for debug
```

Configuration (name)

This function allows the user to apply a configuration to their Environment. Normally the environment passed to Parts would have called this already, however users who are not using this Parts feature, or who need to create a custom environment setup for some reason can call this to modify or apply new settings. This function should be called after the `ToolChain()` function to make sure that settings for the configured tools are correctly loaded and applied. Note that this is an advance function, as currently there is no way to have passed or get in a Part file a basic environment with the entire standard Part variables added to it. You can only get a Raw SCons environment. If you use this in a normal Parts file the configuration will be added to the currently defined one. The custom configuration file for a given tool however can be defined for a given tool to replace all value as a workaround.

Arguments:

- `name` – is the name of the configuration you want to apply.

Access as:

- Method of SCons Environment

Example:

Apply configuration called 'myconfig'

```
env.Configuration('myconfig') :
```

Apply release configuration

```
env.Configuration('release'):
```

Platform Handling

Part extends the current platform handling of SCons by adding information about the general architecture. This information can be found in variable defined in the environment object as well from a new SystemPlatform object that represents the OS-ARCH combination. In future version this may be extended to add new items about the system that are generally useful to building and cross building.

Environment Variables

Currently SCons uses a value of PLATFORM to define if the host system is win32, posix, darwin, etc. When cross building this information is not helpful as you are often compiling a different system. Likewise information about the architecture you are compiling for, such as x86, or arm is needed to step up a tool chains and certain settings to build correctly. To help with this six new values are added to the environment by Parts.

- HOST_OS – defines the current OS platform we are building on. This is the same as the current PLATFORM value in SCons.
- TARGET_OS – defines the current OS platform we are building for
- HOST_ARCH – defines the current architecture of the system we are building on
- TARGET_ARCH – defines the current architecture of the system we are building for
- HOST_PLATFORM – This is the SystemPlatform object of the OS and Architecture of the system hosting the build. This value is ideally read only.
- TARGET_PLATFORM – This is the SystemPlatform object of the OS and Architecture of the system we are building for. If set at correct location, it will control what the build will build target. It can be set via --target command line argument, or via the SetOptionDefault() api to control the default build target. It can be set by passing the value in to a Part() call to control how that given Parts should cross build. This is useful in cases when you want a one pass build that cross builds a set of component vs. have to do two separate passes.

Adding new platforms

There is a developer API (and hopefully soon a user level API) to allow the addition of new platforms, and defining mappings to existing platform with different values. This allow easy extension of the system to new undefined systems, or system that have not been added yet to Parts (and SCons). Merging of different values for predefined systems to different value that can help when migrating from a different build system can also be done, such as mapping AMD64 to x86_64. If Parts does can't map the input to an existing platform it will error out.

SystemPlatform(os,arch)

Parts defines a object called SystemPlatform. This object is basically a way to deal with all System Platform information with a single object. This allow for safe testing and cleaner code. Currently the concept of the System Platform is a combination of OS and general Architecture. This in the future may also deal with OS Vender and CPU. Currently in Parts HOST_PLATFORM is used to define the value of system hosting the build and TARGET_PLATFORM for the system we are targeting the build for.

Arguments:

- `os` -- this is the OS value of the system. It should be a string. A value of **'any'** is used as a wildcard
- `arch` -- this is the architecture value of the system. It should be a string. A value of **'any'** is used as a wildcard

Access as:

- Object in SConstruct file
- Object in Parts file

Examples:

Create a SystemPlatform object

```
SystemPlatform('posix', 'x86_64')
```

Create a SystemPlatform object that targets a different architecture

```
SystemPlatform(arch='x86_64')
```

Create a SystemPlatform object with only a string

```
SystemPlatform('posix-x86_64')
```

Create a SystemPlatform object wildcard for any architecture of a windows platform

```
SystemPlatform('win32', 'any')
```

Have a parts build 32-bit and 64-bit

```
Parts("mypart32", "mypart.parts", TARGET_PLATFORM='x86')  
Parts("mypart64", "mypart.parts", TARGET_PLATFORM='x86_64')
```

Attributes:

- `OS` -- the value of the OS.
- `ARCH` -- the value of the architecture.

Methods

`SystemPlatform.operators ==, !=`

Allows the testing a SystemPlatform with a string and or SystemPlatform object for easy testing and clean code.

Defines test if target platform is POSIX 64-bit

```
env['TARGET_PLATFORM'] == 'posix-x86_64'
```

Defines test if target platform is POSIX 64-bit (another way)

```
env['TARGET_PLATFORM']==SystemPlatform('posix','x86_64')
```

Defines test if target platform is a Intel compatible 32-bit architecture

```
env['TARGET_PLATFORM']=='any-x86'
```

Defines test if target platform is a Intel compatible 32-bit architecture (another way)

```
env['TARGET_PLATFORM']==SystemPlatform('any','x86')
```

Defines test if target platform is a Intel compatible 32-bit architecture (yet another way)

```
env['TARGET_PLATFORM']=='x86'
```

Defines test if this is a windows system we are building for.

```
env['TARGET_PLATFORM']=='win32-any'
```

Defines test if this is a windows system we are building for. (another way)

```
env['TARGET_PLATFORM']==SystemPlatform('win32','any')
```

Defines test if this is a windows system we are building for.

```
env['TARGET_PLATFORM']=='win32'
```

Tool Chains

Tool chains are named sets of tools and settings used to find and configure the runtime environment to run the tool. This extension is to allow for better control of what tools are setup via the command line or other configuration means. This logic should allow a better replacement of the default tool logic built in to SCons as the user can override or define their own logic. This area of the API is still growing. See the section on defining your tool chain for more details

Tool chain API calls

ToolChain(chainlist)

This function, in short, will call the SCons tool function under the covers and setup values needed for a given tool to correctly configure the environment as defined in the file that defines the tool chain. Note that this is an advance function, as currently there is no way to have passed or get in a Part file a basic environment with the entire standard Part variables added to it. You can only get a Raw SCons environment. If you use this in a normal Parts file the toolchain will added to the current default tools which can lead to mixed environments that might not work as expected. See the sections about Part variable to see example on to better way to set a tool chain in Parts.

Arguments:

- chainlist—list of tool chain names to apply

Access as:

- Method of SCons Environment

Examples:

Apply latest installed version of the Microsoft compiler tool chain

```
env.ToolChain(['cl'])
```

Apply the 2005 version (8.0 version) of the Microsoft compiler tool chain

```
env.ToolChain(['cl_8.0'])
```

Apply latest installed version of the Microsoft compiler tool chain and Perl

```
env.ToolChain(['cl','perl'])
```

Apply the Intel Compiler 11.1 and use GNU compiler tools 4.1.2

```
env.ToolChain(['icc_11.1','gntools_4.1.2'])
```

Apply the Intel Compiler 11.1 and use Microsoft tools 2005

```
env.ToolChain(['icl_11.1','mstools_8.0'])
```

Node helpers

There are a number of functions to help deal with gather sets of node and dealing with certain annoying path issues in SCons. Dealing with Symlinks on Microsoft Windows (Vista and above), POSIX and Darwin systems.

Node helper API calls

CCOPY(TARGET, SOURCE ,COPY_LOGIC=CCOPY.DEFAULT,**KW)

This method allows copying of files or directories to a certain location. How files are copied can be controlled to allow copies to be done with hard links or symlinks, vs a straight copy to help save disk space. This function is aware of Pattern objects. If there is a problem with certain type of link being made, the user can add – verbose=ccopy for information on what the system is reporting. In general modern system all support hard links. Windows Vista added the needed support to make symlink practical; however you have to make sure that you have set the correct permission for a given user to use then. If the user is an Admin user, make sure the UAC is turned off as this messes up admin user by removing the symlinks rights, which allowing normal user with permissions to work correctly.

Arguments:

- target – This is a directory in which all items in the source will be copied to.
- Source – the sources in which to copy into the target directory
- copy_logic – optional value to control how to do a copy. By default it used the value provided by the --copy-logic argument. Values which can be provided are:
 - CCopy.default – use default logic. This is currently CCopy.hard_soft_copy
 - CCopy.hard_soft_copy – Try to make a hard link, else try to make a symlink, else try to do a full copy
 - CCopy.soft_hard_copy – Try to make a symlink, else try to make a hard link, else try to do a full copy
 - CCopy.soft_copy – Try to make a symlink, else try to do a full copy
 - CCopy.hard_copy – Try to make a hard link, else try to do a full copy
 - CCopy.copy – do a full copy
- Kw – extra values to pass to the builder

Access as:

- Method of SCons Environment

Examples:

Copy files to a directory.

```
env.CCopy("#Special_place", ['foo.txt', 'boo.txt'])
```

*CCopyAs(target, source ,copy_logic=CCopy.default,**kw)*

This method allows copying of files or directories to a certain location while allowing a given item to be renamed or copied in a slight different place. Minus this difference this function is the same as CCopy.

Arguments:

- target – List of File object that match the value in the source list to be copied to with possible different value.
- Source – The list of file objects to copy.
- copy_logic – optional value to control how to do a copy. By default it used the value provided by the --copy-logic argument. Values which can be provided are:
 - CCopy.default – use default logic. This is currently CCopy.hard_soft_copy
 - CCopy.hard_soft_copy – Try to make a hard link, else try to make a symlink, else try to do a full copy
 - CCopy.soft_hard_copy – Try to make a symlink, else try to make a hard link, else try to do a full copy
 - CCopy.soft_copy – Try to make a symlink, else try to do a full copy
 - CCopy.hard_copy – Try to make a hard link, else try to do a full copy
 - CCopy.copy – do a full copy
- Kw – extra values to pass to the builder

Access as:

- Method of SCons Environment

Examples:

Copy files and renames them to different places

```
env.CCopyAs(["readme", "sub1/details.txt"], ['foo.txt', 'boo.txt'])
```

AbsDir(path) or AbsFile(path)

These functions allow creating a SCons Node object that correctly relates to File or Directory above or outside the current directory structure. SCons dislikes the accessing of paths that are relatively above the current location of the SConstruct file or Part file. However it is common to want to include path such as "../include". Currently this gets handled incorrectly which can make using SCons with existing source tree structures difficult. These functions

allow for this to be handled without hard-coding, or dealing with more complex SCons code. The function returns the corresponding node in a form that is seen as an absolute path. In terms of SCons it is in the location of the `src_dir`, not the `variant_dir`. It should be noted that it is a lot better to move your Part file up a level if you can, as this allow the use of certain features in SCons that would have issues with dealing with mapping directory or file nodes to virtual spaces.

Arguments:

- `path` – is the path used for making a SCons File node or Dir node

Access as:

- Method of SCons Environment
- Function in Parts file

Examples:

Add a path to be used to find header files.

```
env.Append(CPPPATH= [AbsDir(' ../include' )])
```

The Pattern object

The Pattern object is much like the SCons `Glob()` function. It was originally design before the `Glob` function was designed and added to SCons. The main difference is that the Pattern object form the `Glob` object is that the Pattern object makes it easy to exclude patterns, and do recursive searches, whereas the `Glob()` does not. When used by API call that can handle it, the pattern object will preserve directory structure. This mean API like `CCopy()`, the `InstallXXX()`, the `SdkXXX()` and `UnitTest()` can easily copy over a tree of data of certain files. Very useful to gathering recursive sets of files in a directory. The known input patterns are in the table below

Pattern	Meaning
<code>*</code>	matches everything
<code>?</code>	matches any single character
<code>[seq]</code>	matches any character in <i>seq</i>
<code>[! seq]</code>	matches any character not in <i>seq</i>

```
Pattern(sub_dir='',src_dir = './', includes = ['*'], excludes = None,recursive=True)
```

ARGUMENTS:

- sub_dir – optional subdirectory used by to tell where files that match the pattern should be copied to, ignored otherwise.
- src_dir – optional directory used as the root for where the search for files should start
- includes – optional list of pattern to use for finding a match
- excludes—optional list of patterns to exclude if the file did match one of the includes patterns

Access as:

- Method of SCons Environment
- Object in SConstruct file

EXAMPLE:

Creates an object that will return all C or C++ source files, but will exclude anything in the ignore directory.

```
Pattern(includes = ['*.cpp', '*.c'], excludes=['*/.ignore*'])
```

Methods:

FILES(DIRECTORY=NONE)

Returns list of files that match the pattern, useful when giving the files to native SCons functions that don't understand Pattern object as a first class object.

ARGUMENTS:

- directory – allow selection of files that match the pattern at a given directory level. If None is given all files are provided

EXAMPLES:

Gather all source files in current directory and below to be built as a shared library. Notice that we have to use files() here because SharedLibrary does not understand Pattern

```
cpp_files=Pattern(includes = ['*.cpp', '*.c']).files()
env.SharedLibrary('mylib', cpp_files)
```

Retrieves all file in the sub-directory “types”

```
Pattern(includes = ['*.cpp', '*.c']).files('types')
```

Copy all file in a current directory to a different location while keeping the directory structure.

```
Env.CCopy("to/location", Pattern())
```

SDKs and Exporting

SDK's and exporting are the primary way Parts shares information between different Parts, such as include paths and libraries to link. Using SDK functions is much like using the Install functions. While the Install functions are for defining what should go in the product packages, the SDK functions define what should go in a SDK. Export functions on the other hand define extra data that is common to building code, such as defines or flags. Much like Install there are a number of methods defined that are based on concept and provide similar inputs. New concepts will be added over time as needed.

The SDK Methods

In general most of the SDK methods behave the same as the Install methods. In a few cases they have differing behavior.

Access as:

- Method of SCons Environment

Declarations:

SDKINCLUDE(SOURCES,SUB_DIR="",ADD_TO_PATH=TRUE,USE_SRC_DIR=FALSE,CREATE_SDK=TRUE)

Adds the headers files to the SDK as well as adds the path to the CPPPATH for any dependent component.

SDKLIB(SOURCES,SUB_DIR="",ADD_TO_PATH=TRUE,AUTO_ADD_LIBS=TRUE,USE_SRC_DIR=FALSE,CREATE_SDK=TRUE)

Adds the files to the SDK and adds the files to the LIBS variable and adds the path of the file to the LIBPATH for any dependent component. On POSIX LIBPATH setting will also be added to a -Wl,-rpath-link option.

SDKBIN(SOURCES,SUB_DIR="",CREATE_SDK=TRUE)

Adds the file to the SDK bin directory, doesn't add anything to dependent Parts

`SDK(SOURCES,SUB_DIR="",ADD_TO_PATH=TRUE,AUTO_ADD_LIBS=TRUE,USE_SRC_DIR=FALSE,CREATE_SDK=TRUE)`

`SDKTARGET(SOURCES,SUB_DIR="",ADD_TO_PATH=TRUE,AUTO_ADD_LIBS=TRUE,USE_SRC_DIR=FALSE,CREATE_SDK=TRUE)`

.....

Sdk and SdkTarget are the same method. Originally this was called Sdk, but SdkTarget was added latter to map to InstallTarget, as Install couldn't be used as it clashed with the existing SCons Install method. This method will sort files based on whether they should be put into the lib or bin directory. Unknown file types are ignored, however programs have no file extension on POSIX system, so if the file is not a library type it is assumed to be placed in the bin directory.

`SDKCONFIG(SOURCES,SUB_DIR="",CREATE_SDK=TRUE)`

.....

Called for files that are considered to be configuration files.

`SDKDOC(SOURCES,SUB_DIR="",CREATE_SDK=TRUE)`

.....

Called for files that are considered to be documentation files.

`SDKHELP(SOURCES,SUB_DIR="",CREATE_SDK=TRUE)`

.....

Called for files that are considered to be help files.

`SDKDATA(SOURCES,SUB_DIR="",CREATE_SDK=TRUE)`

.....

Called for files that are considered to be random data files.

`SDKMESSAGE(SOURCES,SUB_DIR="",CREATE_SDK=TRUE)`

.....

Called for files that are considered to be message catalog files.

`SDKRESOURCE(SOURCES,SUB_DIR="",CREATE_SDK=TRUE)`

.....

Called for files that are considered to be resource files, classically files for GUI's such as images and other GUI related resources.

`SDKAPI(SOURCES,SUB_DIR="",CREATE_SDK=TRUE)`

.....

Called for files that are considered to be part of an installable API

`SDKSAMPLE(SOURCES,SUB_DIR="",CREATE_SDK=TRUE)`

.....

Called for files that are considered to be part of a sample

SDKTOPLEVEL(SOURCES,SUB_DIR="",CREATE_SDK=TRUE)

Called for files that need to be installed, or added in the root directory of the final product. This call allows us to add them to a SDK.

SDKNOPKG(SOURCES,SUB_DIR="",CREATE_SDK=TRUE)

Called for files that are needed by the final installer package, but are not going to be installed on the user system. Useful for items such as scripts or programs that may run as part of the install or setup process, but are not added to the final install bits. This call allows us to add them to a SDK.

Arguments:

- `sources` – list of Files as strings, SCons Node or Parts Pattern objects to be added to the SDK
- `sub_dir` – the subdirectory under the SDK concept to add these file to.
- `add_to_path` – this value controls the `sub_dir` that should be added to the corresponding path. For `SdkInclude` this controls whether the `sub_dir` path is added to `CPPPATH` while for `SdkLib` whether it is added to `LIBPATH`. The base SDK path is always added. Defaults to True.
- `auto_add_libs` – this value controls whether the file should be added to the `LIBS` variable of dependent Parts. Defaults to True. Useful for cases when many different libraries may be outputted and it is better for the user to explicitly define which lib to use.
- `use_src_dir` – this tells Parts to use the `src_dir` location path instead of the SDK path. This is useful for cases when headers can't be properly parsed, so the dependence information SCons would create is incomplete, or for debugging as the debugging information will point to the original file instead of the file in the SDK.
- `create_sdk` – This value controls if this line will be added to the generated SDK file. This value is True by default. It is used in the generated SDKs. Setting to False is useful when defining a Part that has many outputs, where some outputs are not wanted in the final SDK . For example the outputs might be internal and not needed to build against, These file will still be copied to a SDK directory, only the entries in the generated SDK file will be missing

Examples:

Easiest way to add outputs from a SharedLibrary or a Program call to the SDK.

```
outputs=env.SharedLibrary(binary,cpp_files)  
env.Sdk(outputs)
```

Easiest way to copy a group of headers to the SDK.

```
env.SdkInclude(Pattern(includes = ['*.hpp', '*.h']))
```

Copy a config file to subdirectory

```
env.SdkConfig(["system.cfg", "core"])
```

The Export Methods

These methods are used for exporting values that are useful to share such as paths, defines and tool flags. Value such as flags will need to be explicitly requested in the dependent components via the REQ flags in the “requires” argument of the Component object.

Access as:

- Method of SCons Environment

Declarations:

EXPORTCPPPATH(VALUE,CREATE_SDK=TRUE)

Exports values to CPPPATH value. This is more fine grained control over the path than using SdkInclude. Unlike SdkInclude there will be no files copied to SDK path.

EXPORTCPPDEFINES(VALUE,CREATE_SDK=TRUE)

Exports values to CPPDEFINES

EXPORTCFLAGS(VALUE,CREATE_SDK=TRUE)

Exports values to CFLAGS

EXPORTCCFLAGS(VALUE,CREATE_SDK=TRUE)

Exports values to CCFLAGS

EXPORTCXXFLAGS(VALUE,CREATE_SDK=TRUE)

Exports values to CXXFLAGS

EXPORTLINKFLAGS(VALUE,CREATE_SDK=TRUE)

Exports values to LINKFLAGS

Arguments:

- sources – list of values to be exported

- `create_sdk` – this value controls if this line will be added to the generated SDK file. This value is true by default. In normal use this shouldn't be used.

Examples:

Export a define to help users build against this library

```
env.SdkInclude(Pattern(includes = ['*.hpp', '*.h']))  
  
env.ExportCPPDEFINES(['MY_LIB_AS_STATIC'])
```

Packaging

A number of API have been added to help with the packaging of the final bit in to some sort of Install Package. The API is still being developed, because of it is incomplete and still subject to improvements with tweak to existing API and addition of new one. The end goal of these API is to make it easy for a product owner to control layout of different type of files, such as configuration file, or programs or shared libraries, independent of what a given Parts needs to have installed and in a mostly platform neutral way. To achieve this, a few different sets of API's exists to help define different aspects of what should be packaged

Install sandbox

The install sandbox is an area in which all outputted files will be put to allow running of all the Parts as if it was an installed product. The install functions in general wrap the built-in SCons Install function, while adding extra information with `MetaTag()` is added to each node. Currently the Install API is defined in a form to try to express concepts of what to have installed. This allows at a higher level for product managers to define where a given file goes. This also allows component owners, which have their code shared between different products, to be abstracted from where their files are installed on disk. The api does allow component owners to control what concept a file is installed under, and any sub-directories that might be needed for more advance cases. Other concepts will be added over time as needed or requested.

The Install API

In general, most of the Install functions behave the same. In a few cases they have differing behavior.

Access as:

- Method of SCons Environment

`InstallTarget(src_files, sub_dir='', no_pkg=False, create_sdk=True)`

Sorts the input files via file extensions to determine where they should go. Currently only handles libs and program files. Because POSIX systems don't have a program extension it is impossible to correctly tell via string name alone if the file is a Program file or not. Because of this the current logic is, if it not anything else, assume it is a program. On windows, DLLs are copied to the InstallBin area. On POSIX systems, .so files are copied to the InstallLib area.

```
InstallBin(src_files, sub_dir="", no_pkg=False, create_sdk=True)
```

Install files into the defined INSTALL_BIN directory. Ideally for all binaries that are programs.

```
InstallLib(src_files, sub_dir="", no_pkg=False, create_sdk=True)
```

Install files in to the defined INSTALL_LIB directory. This is ideally for all library files.

```
InstallConfig(src_files, sub_dir="", no_pkg=False, create_sdk=True)
```

For configuration file

```
InstallHelp(src_files, sub_dir="", no_pkg=False, create_sdk=True)
```

For help files

```
InstallDoc(src_files, sub_dir="", no_pkg=False, create_sdk=True)
```

For documentation as documentation is defined differently from help in a product.

```
InstallMessage(src_files, sub_dir="", no_pkg=False, create_sdk=True)
```

For the translated message catalog files for a product.

```
InstallResource(src_files, sub_dir="", no_pkg=False, create_sdk=True)
```

Resources such as GUI resource bundles

```
InstallAPI(src_files, sub_dir="", no_pkg=False, create_sdk=True)
```

For items that need to be added to an API

```
InstallSample(src_files, sub_dir="", no_pkg=False, create_sdk=True)
```

For items that are parts of samples shipped with the product

```
InstallData(src_files, sub_dir="", no_pkg=False, create_sdk=True)
```

For items that need to be added to a generic data location.

```
InstallTopLevel(src_files, sub_dir="", no_pkg=False, create_sdk=True)
```

This is for installing files in the top level area of the product. This is the classic area for EULA's and readme.txt documents

PkgNoInstall(src_files, sub_dir="", no_pkg=False, create_sdk=True)

These are for files needed for the install builders that will not be installed, but would be used by custom command via the installer during the install.

Arguments:

- `src_files` – the list of files to be installed. Can be File nodes, strings or Pattern objects.
- `sub_dir` – sub directory to install these files or tree structure (if a pattern object was used) under
- `no_pkg` – This value, if set to True, will prevent the source files from being added to the global list used by SCons Package functions. This allows items to be added to the install sandbox but not added to the final package file.
- `create_sdk` -- This value controls if this line will be added to the generated SDK file. This value is True by default. It is used in the generated SDKs. Setting to False is useful when defining a Part that has many outputs, where some outputs are not wanted in the final SDK . For example the outputs might be internal and not needed to build against, These file will still be copied to a SDK directory, only the entries in the generated SDK file will be missing

Examples:

Installs a file under the bin directory

```
env.InstallBin(["mydata.bin"])
```

Installs a set of files to be sorted and installed

```
outputs=env.SharedLibrary(binary,cpp_files)
env.InstallTarget(env.SdkTarget(outputs))
```

Package Groups

When creating an install package it is common to separate files in to different groups. This is primarily done to help control which set of files may get installed based on certain runtime condition, such as certain script checks about the system being installed on, or based on user input to control which set of features get installed. In other cases different types of install packages are created which may need different set of files depending on what is needed for a given install package, or different output formats may prefer handling certain sets of files. For example on Windows system it is not uncommon to install 32-bit and 64-bit binaries based on user input from a single MSI file, which on Linux it is often the case that 32-bit and 64-bit binaries are in separate RPM, or DPKG files. Package groups allow high level control over how certain files are grouped and handled in during packaging.

The Package Group API

These APIs allow the user to map a Part of a node to a given package group, and provides abilities to get which nodes are mapped to a given package group. The easiest way however to install map a package group to a Part is via passing in `package_group="groupvalue"` to the Parts call. For example:

```
Part("gui.part",package_group="gui-{$TARGET_PLATFORM}")
```

This would map this Part to a **gui-win32-x86** if building for a windows 32-bit system or **gui-posix-x86_64** group if building for a POSIX 64-bit system. Depending on what you are building for it might be something else.

PackageGroups()

Returns all defined package groups as a list of strings. The value returned may differ over time as new package groups may be defined over time.

Access as:

- Function in Parts file
- Functions in Sconstruct

Argument:

None

Examples:

Get all known package groups.

```
ver=PackageGroups()
```

PackageGroup(name,parts)

Adds one or more Parts to a given package group

Access as:

- Function in Parts file
- Functions in Sconstruct

Argument:

- Name – the package group we want to add to
- Parts – a list of Parts Aliases to map. This API will be updated to in future drops to take better inputs.

Examples:

Map “parta” to package group with the name of “gui-group”.

```
PackageGroup("gui-group", [ 'parta' ])
```

PackageGroup(name)

Returns all Parts mapped to the provide group. This value is dynamic and will change until all Parts file are loaded.

Access as:

- Function in Parts file
- Functions in Sconstruct

Argument:

- Name – the package group we want to add to

Examples:

Get all parts mapped to gui-group

```
ver=PackageGroup("gui-group")
```

GetPackageGroupFiles(name,no_pkg=False)

Returns all output files mapped to the provide group. This value is dynamic and will change until all Parts file are loaded.

Access as:

- Function in Parts file
- Functions in Sconstruct

Argument:

- Name – the package group we want to add to
- no_pkg– Items can be installed with no_pkg value meaning that these should not be added to the install package for some reason. Setting this to True allows you to get a list of all items that that would have been map, if this value was not set in the Install call.

Examples:

Get all files mapped to gui-group

```
ver= GetPackageGroupFiles ("gui-group")
```

The Package Group Criteria API

These functions allow the user to provide a set of callback function that will be run on every node to remap it to a different group. This allows for cases when a Part might install some binaries and configuration files and one needs to sort all the configuration files in to a different group from the binaries. This allows a high level way to redirect the installed node into separate groups. The order in which the callback are called does matter, as soon as a node has been remapped, it is set and any other callback that have been registered will no called on the given node.

ReplacePackageGroupCriteria(name,func)

This function will replace all currently defined functions with the list of passed in functions callbacks

Arguments:

- name – The name of the package group to assign a node to if the passed in callback functions returns True
- func – a single callback function or a list of call back functions to test is a given node should be remapped.

Access as:

- Function in SConstruct file

Examples:

Remap all configuration files, installed with InstallConfig() to the “cfg_files” package group.

```
ReplacePackageGroupCriteria('cfg_files',lambda node:  
MetaTagValue(node,'category','package')== 'CONFIG')
```

AppendPackageGroupCriteria(name,func)

This function will append the currently defined functions with the list of passed in functions callbacks

Arguments:

- name – The name of the package group to assign a node to if the passed in callback functions returns True
- func – a single callback function or a list of call back functions to test is a given node should be remapped.

Access as:

- Function in SConstruct file

Examples:

Map a callback to remap all resource files, installed with InstallResource() to the “cfg_files” package group.

```
AppendPackageGroupCriteria('cfg_files', lambda node:
MetaTagValue(node, 'category', 'package') == 'RESOURCE')
```

PrependPackageGroupCriteria(name,func)

This function will prepend the currently defined functions with the list of passed in functions callbacks

Arguments:

- name – The name of the package group to assign a node to if the passed in callback functions returns True
- func – a single callback function or a list of call back functions to test if a given node should be remapped.

Access as:

- Function in SConstruct file

Examples:

Map a callback to remap files that ends with .cfg to the “cfg_files” package group.

```
AppendPackageGroupCriteria('cfg_files', lambda node:
node.path.endswith('.cfg'))
```

CallbackFunction(node)

The call back function will be passed a node, and should return True if it should be remapped, False if it should not.

Arguments:

- node – The name of the package group to assign a node to if the passed in callback functions returns True

Access as:

This is user defined

Examples:

A callback that will remap if the current package group startswith “gui-” and matches a certain category of files being installed.

```
def gui_filter(node,packge_cat):  
    ret=(MetaTagValue(node,'category','package')==packge_cat and  
        (MetaTagValue(node,'group','package').startswith('gui-')))  
    return ret  
  
...  
  
# map all files from Parts mapped to a GUI named group to  
# a common GUI group if the files have been installed as  
# Configuration or message catalog files.  
ReplacePackageGroupCriteria('gui_common-${TARGET_OS}-any',  
                             [lambda x : gui_filter(x,'CONFIG'),  
                              lambda x : gui_filter(x,'MESSAGE')])
```

Version control system

The version control system allows for the ability to retrieve source trees from different sources. This currently is different from the SCons SourceControl concept is that it will not try to update the source for ever build, and it will not update one source at a time put the whole directory tree. Vcs logic in Parts will try to update the code based on a policy that can be set, where the default is to check out the source is the code does not exist or switch/update if the repository location does not match the location provided in the VCS call (ie what happens when you switch branches). There is an --update options to allow manual control over when to update the source locally for possible changes that had been checked in.

The design allows for as many VCS object types to be defined as needed along with the ability for a user to customize behavior of newly defined objects. For example, defining a VCS object that would allow for predefined knowledge of how source maybe laid out in the SVN tree so that grabbing the latest stable source is an automatic operation.

Vcs API

`VcsSvn(repository,server="",revision=None)`

This object allows access to a SVN repository.

Arguments:

- `repository` — this is the SVN directory we want to check out from.

- server – this is the SVN server we want to check out from, if not provided it will use the value of \$SVN_SERVER if defined.
- revision – this is the revision of SVN to checkout, if not defined it will use the value of \$SVN_REVISION or if that is not set then it will use the latest revision

Access as:

- Function in SConstruct file

Examples:

Check out some Parts from SVN

```
SetOptionDefault('SVN_SERVER','file:///'+path)

Part('proxy.parts',vcs_type=VcsSvn(repository='proxy/',server='https://'+
'SConsrocks.org'))

Part('engine.parts',vcs_type=VcsSvn(repository='engine/'))

Part('src/print_msg.parts',vcs_type=VcsSvn(repository='print_msg/'))
```

`VcsFileSystem(repository,server="")`

This allows copying and syncing data from a local or remote file system.

Arguments:

- repository — this is the directory we want to copy from.
- server – this is the server or root directory we want to copy from. If not provided it is ignored, otherwise it is prepended via `os.path.join()` to repository value

Access as:

- Function in SConstruct file

Examples:

Check out some Parts from SVN

```
Part('foo.parts',vcs_type= VcsFileSystem(repository='c:/foo'))
```

VCSREUSE (PART)

This allows the ability to use a previous Parts' checkout information to access the Parts file. This allows single Checkout of a tree that might have multiple Parts files in it reducing checkout time and saving disk space. It can also be used to control multiple builds of the same part with different setup values, such as build the Parts 32 and 64 bit in the same build pass.

Arguments:

- part — this is the part object, returned from the Part call, or a string referring to the part

Examples:

Check out some Part from SVN and reuse it for other Parts files in the same Source Tree.

```
P=Part('proxy/proxy.parts',vcs_type=
VcsReuse(repository='tree',server='https://'+SConsrocks.org'))

Part('engine','engine/engine.parts',vcs_type=VcsUsePriorPart(p))

Part('print_msg','print_msg/src/print_msg.parts',VcsUsePriorPart(p))
```

Cross build a Part 32-bit and 64-bit, but reuse the same checkout area.

```
P=Part('proxy/proxy.parts',vcs_type=VcsSvn(repository='proxy',server='http://'+SConsrocks.org'),TARGET_PLATFORM='x86')

Part('proxy/proxy.parts',vcs_type=VcsUsePriorPart(p,TARGET_PLATFORM='x86_64'))
```

Testing

Testing is an important aspect of any project. Parts attempts to define a set of functions to help with different common testing concepts, that may be needed to help produce a quality product release. Currently only the concept of unit testing has been tackled. In future other testing concepts maybe attempted and added to handle

more high level testing ideas. All testing concepts will get an independent set of aliases to allow better control over when and how they are built.

UnitTest method

The UnitTest method allows handling of common unit testing needs. Unit testing is the most basic form of testing. The way it is generally defined in Parts is a set of file that will compile into a program and run. Unit testing often requires special environment, command line or data file to run correctly. To this end, arguments and variable have been defined to help control these values. For instance data file will be copied to a special unit testing sand box location, and the given test will run as if it is in that directory to make it easy for the test to find the data file without special knowledge of the directory layout of the sandbox. Scripts are auto generated to allow running the test outside SCons correctly and to help in debugging, or use with existing testing infrastructure.

Access as:

- Method of SCons Environment

Declarations:

```
UNIT_TEST(TARGET, SOURCE, COMMAND_ARGS=[], DATA_SRC=[], SRC_DIR='.',  
MAKE_PDB=TRUE, DEPENDS=NONE,BUILDER="PROGRAM",BUILDER_KW={}, **KW):
```

- target – the name of the test
- source – a list of files or pattern objects to be used to create the test program
- command_args – optional list of values to pass to the test program when it runs. These values will be expanded via env.subst
- data_src – optional list of pattern objects or files that are needed as inputs to run the unit tests correctly. Useful for tests that test parsers or data processing of an API
- src_dir – this optional value is useful when the unit tests exist in a directory above the current directory of the Part file.
- make_pdb – only for windows platforms; controls if a PDB should be created as Part of the test to help with debugging. The default is true.
- depends – Takes options list of Component objects (Like the DependsOn() api) to custom dependency mappings for the advance unit test usage.
- Builder – name of builder to use to create the unit test program. Defaults to Program() builder which makes a executable to run.
- builder_kw – Optional set of extra keyword arguments to pass to the builder minus the standard sources and targets values.

- ****kw** – optional value that one would want to add to the environment to control what and how the test is built or run.

Examples:

Defines a simple test

```
env.UnitTest("test", ["test.cpp"])
```

Defines a number of tests

```
env.UnitTest("startup", ["startup_test.cpp"])
env.UnitTest("common", ["common_test.cpp"])
env.UnitTest("node", ["node_test.cpp", "node_test2.cpp"])
env.UnitTest("tool", ["tool_test.cpp"])
```

defines a test that uses TUT headers in a relative directory ../tut, affects only the UnitTest call

```
p=Pattern(includes = ['*.cpp', '*.c'])
env.UnitTest("test", [p], CPPPATH=[AbsDir('../tut')])
```

another way to say the same thing, but affects all UnitTest calls called via the env environment.

```
p=Pattern(includes = ['*.cpp', '*.c'])
env.Append(CPPPATH=[AbsDir('../tut')])
env.UnitTest("test", [p])
```

builds some test that exists in a relative directory of "../utest"

```
p= Pattern(src_dir = '../utest', includes = ['*.cpp', '*.c'])
env2.UnitTest('test', [p, 'main.cxx'], src_dir='../utest')
```

adds some data file that would be used to help the tests run. The data_ptn would the XML and txt files over, with the tree structure intact from the directory "../utest/inputs". The file ibuild_ref.cfg would be expected to exist in ../utest.

```
p= Pattern(src_dir = '../utest', includes = ['*.cpp', '*.c'])
data_ptn=[Pattern(src_dir='../utest/inputs',
                  includes = ['*.xml', '*.txt'])
env2.UnitTest('test', [p, 'main.cxx'], src_dir='../utest',
data_src=[data_ptn, 'ibuild_ref.cfg'])
```

Add command line arguments to the test

```
env.UnitTest("test",["test.cpp"], command_args=['-verbose'])
```

Similar to the above sample, but add this value to the shell environment.

```
env.UnitTest("test",["test.cpp"], UNIT_TEST_ENV={'VERBOSE':True})
```

Python script runner

This function warps a builder object that allows the execution of an external python script. This allows the user to address two issues when using Command/Execute or direct importing of the script. In the case when the script has the same name in many different Parts, but the content of the script is different, this function allows these scripts to be sandboxed from each other in the python loader, without this random behavior can happen as Python will think that it has already imported a certain script. If a custom builder object was made to execute a function in the script random behavior can happen as the load order and use of import reloading can effect what the final builder will do when it is called. This can be very difficult to debug and correct in case of parallel builds. The other case this helps solve is providing an easy way to make sure your script is called only when it has to be during the build and providing a standard way to get data about what the script will do. Without this it can be difficult to tell SCons what the script should provide during the read phase, so SCons can call it correctly during the build phase.

To deal with this the python script will load a script with a special importer to prevent issues of script overriding. The script will have to provide two functions. One will need to return a list of files that will be created when the script is ran, and the other will do the actual actions needed to create these files. The names of these function by default will be emit and build, but the user can override what entry points to use.

Python script runner API

```
PythonScript(file,emit_func='emit',build_func='build',**kw)
```

Arguments:

- `emit_func` – this is the name of the function to be called for returning the list of targets. By default it is called `emit`, it is passed `**kw`.
- `build_func` – this is the name of the function to be called for building the list of targets. By default it is called `build`, it is passed `**kw`.
- `**kw` – these are the extra named parameters that will be passed to the `emit_func` and `build_func`. It is passed in the form of `func(**kw)`.

Access as:

- Method of SCons Environment

Examples:

Call scripts that will generate some header files.

```
headers=env.PythonScript('gen_headers.py')
```

The gen_headers.py script has these function defined in it

```
def emit(**kw):  
    return make_list() # create list of files that will be made  
  
def build(**kw):  
    return gen_headers() # actually make them.
```

Documentation building

TDB

Configuring Parts

Parts can be configured in a number of different ways, via the command line option, variable that can be set on the command like, in a file or via API calls. Extension via the parts-site feature can be used to set and define values as well. Work is still ongoing to make configuring Parts more stream lined and consistent, because of this changes will happen that might not be 100% backward compatible.

Command line vs. Variable

SCons allow for new command line options to be defined in a --options form, or via a variable=value form. The command line form advantages are:

- Easier to validate input value
- Misspelled options are flags as error, not just passed through
- Generally better error handling support if input value are incorrect.

The main advantage of using the variable=value method is:

- Value can be set via Api calls in Sconstruct, or set in a configuration file, or set on the command line, where option on the command line can override values in the configuration file, or Sconstruct, while the configuration file can override value in the Sconstruct. This allows an easy way to set common default, while allowing an easy customization and overriding
- Values can be objects when set in a configuration file or a Sconstruct allowing more advance setup or customization.

- Variables can set and passed to individual Parts or environments when more fine-tuned behavior is needed.

Parts has been moving to define most command options that control behavior or do actions in the command line as this can be better verified and has generally better error handling. Variable that define values such as paths, or define more advance customization via API calls will exist in the variable form. Many variables are also defined to a mapping between a certain command line options. Going forward a new Setting will be added that should hopefully streamline these methods.

Variable Usage

SCons by design allows users to define a command line variable in the form <Key>=<Value>. By using the SCons ARGUMENTS object the user can query for any value added on the command line. Parts as an extra feature will add these values to the environment passed to your Parts file by default. For example:

```
scons myvar=hello
```

will add a variable of myvar with the value of the hello to the env object passed to the parts file. This data can be accessed via code such as:

```
env.get('myvar',None) # return the value of myvar if it exists or None
```

Order of importance

Parts defines an order of importance of which value is used based on the source, the value comes from. The order or precedence is command line, configuration file, and SetOptionDefault(). This allows the ability to easily share configuration values via different means and a useful way to override behavior to customize the build process to meet the developer needs.

Configuration File

The configuration file tries to allow similar syntax to the command line. It is however a python file in implementation. This means that many values will need to have added quotes. Normally these quotes can be added on the command line as well, and may be needed to handle values with spaces or other values that may have special meaning to the command shell you may be using. Parts will look for the file defined by the option - cfg_file=<file>. If this option is not used Parts will then look in the current directory for a file called "parts.cfg". If the configuration file is found the file will be loaded and any variables will be added to the system.

A simple example might look like this:

```
#parts.cfg  
  
config='debug'  
  
toolchain='icl,mscrt_8.0'
```

In this example by saying on command line "scons foobar" one would build the foobar target with the Intel compiler using vc 8 runtime and compile compatibility.

An advance sample might look like this:

```
#parts.cfg

def getSiteSvnServer():

    ...

SVN_SERVER=getSiteSvnServer()
```

In this example the configuration file would automatically set the SVN server depending on the work site used. This way a simple configuration file can be passed around or checked in to allow a team of people to access the best SVN server for their Site location. There are other ways to use configuration files, such as controlling where code checks out on your system, or the location in which code will build. This allows for simple to complex control over how the build system will work helping control resources and functional needs that happen during development on different systems.

Command Line Usage

Parts adds a number of command line options to better control a number of different aspects that Parts extends in SCons. Below is a list of existing options:

Output Options:

`--console-stream=CONSOLE-STREAM`

Control how the console stream is mapped. This is general not useful, except for internal testing of Part itself.

Arguments

Values can be:

- none – Don't output anything
- tty or con: – map to the console device (default)
- stdout – Map to the stdout stream
- stderr – Map to the stderr stream

Examples:

```
Scons all --console-stream=none
```

Hide all output to the console device

`--enable-color, --use-color, --color`

Control the colors that are used for a given stream.

Arguments

Simple values can be:

- true – Use default colors
- false – disable colors. Same as saying `--disable-color`
- full or darkbg – set all streams to a color that is dark background friendly. Yellow is used for warnings and this does not show well on white background.
- Simple – set the stream colors for warning, error, verbose and tracing streams only

More advance control values can be set in the form of `<stream>=<foreground>:<background>],...`

Stream values are:

- `stdout` -- 'o','out','stdout' – the text that results for a cli tool running such as the compiler
- `stderr` -- 'e','err','error','stderr' – the text that goes to the stderr stream or as some text to suggest that it is an error message
- `stdwrn` -- 'w','wrn','warning','stdwrn' – this is the text that has some values to suggest it is a warning message
- `stdmsg` -- 'm','msg','message','stdmsg' – this is the text that is normally outputted by Scons or Parts related to actions going on. Can be turned off with `-s` switch
- `stdverbose` -- 'v','ver','verbose','stdverbose' – this is text outputted based on verbose group provided to the `--verbose` option.
- `Stdtrace`

Possible color values are :

- '0','black','blk'
- '1','red','r'
- '2','green','g'
- '3','yellow','y'
- '4','blue','b'
- '5','purple','magenta','p','m'
- '6','aqua','cyan','a','c'
- '7','white','lightgrey','w','lg'
- '8','gray'
- '9','brightred','br'
- '10','brightgreen','bg'
- '11','brightyellow','by'
- '12','brightblue','bb'
- '13','brightpurple','brightmagenta','bp','bm'
- '14','brightaqua','brightcyan','ba','bc'
- '15','brightwhite','bw'
- 'default' – this the default color being used by the current console window
- 'bright','bold' – this is the brighter version of the color being used by the console, given that it is not already that brighter color
- 'dim' – this is the version of the color being used by the console, given that the current color is not already dim

Examples:

```
Scons all --color=o=r:b,m=g,e=p
```

Set the stdout color to red on blue, the message text color to green and the stderr text to purple

```
--disable-color
```

Turns off all coloring.

Arguments

No arguments are allowed

Examples:

```
Scons all --disable-color
```

No coloring will be used.

--verbose=[VERBOSE]

Control the level of detailed verbose information printed

Arguments

No arguments are required, this is the same as passing the magic value of “all”. The magic value of “all” will print every verbose message defined. A comma separated list of verbose groups can be passed instead to help filter the output. Currently there is no complete defined list of what groups exist, however any verbose message is in the form of:

Verbose: [<group id>] <message>

Print all messages and looking for a group id value that is of value. Then the next run will print only these values.

Some common values that are of use are:

- loading – prints information about what is being loaded.
- update_check – print information about what is viewed note up-to-date and why.
- vcs_update – print information about what is going to be updated from vcs and why.

Examples:

```
Scons all --verbose
```

Print a lot of extra information about what is going on

```
Scons all --verbose=loading,update_check
```

Print information about what is being loaded and what Parts thinks is out of date.

--trace=TRACE

Control the level of trace information printed. This is like the verbose option, however trace information tend to be useful for debugging certain issues and print lots of data out.

Arguments

A comma separated list of trace groups to print out.

Examples:

```
Scons all --trace=mapper
```

Prints out a lot of data about what is happening turning data mapping of information between different Parts.

--log=LOGGER

Set the if a logger is to be used, and possible what type of logger to use.

Arguments

Values can be:

- no value – same as passing true
- true – turn the default logger on
- false – do not log anything
- text – A simple text logger of what when to the screen. This is the default logger.
- Html – this is a basic html logger that captures color of the output based on the default color scheme
- Rtf – a beta rich text file logger
- <user provided value> -- this can be the value of a user defined logger found in the parts-site/logger directory. For example the user might make a rich html logger named rich_html.py. The value provided as an argument would be “**rich_html**”

Examples:

```
Scons all --log
```

Turn on logging using the default logger

```
Scons all --log=text
```

Turn on logging using the text logger

```
Scons all --log=mylogger
```

Turn on logging using the user provided logger name “mylogger”. Would have a mylogger.py file defines in a parts-site/loggers directory.

```
--show-progress=SHOW_PROGRESS
```

Controls if progress state is shown, and how it might be shown

Arguments

Currently the only values supported are:

- True – show progress (the default behavior)
- False – hide progress.

Examples:

```
Scons all --show-progress=false
```

Hide progress from being shown

```
--hide-progress
```

Controls if progress state is shown

Arguments

No arguments are allowed

Examples:

```
Scons all --hide-progress
```

Hide progress from being shown

Build Options:

```
--target=TARGET_PLATFORM, --target-platform=TARGET_PLATFORM
```

By default this value is the value defining the host system. Setting this value allows one to control cross building for a different platform, such as different architecture **x86** vs. **x86_64** or different system such as **android-x86**. Not all tools support cross building for different targets. This maybe because it is not supported by the tool, information on how to setup the tool to cross build has not been defined in Parts or the tool needed to cross build is not installed.

Arguments

Must be a string defining a validate platform known by the build system.

Examples:

```
Scons all --target=x86
```

This builds a 32-bit x86 output.

```
Scons all --target=win32-x86_64
```

This builds a 64-bit windows output

```
Scons all --target=android-x86
```

This builds a 32-bit x86 based android binary output

```
Scons all --target=android-arm
```

This builds a 32-bit arm based android binary output

```
--tool-chain=<tool>[,<tool>], --tc=<tool>[,<tool>]
```

Tells what tool chain should be setup by default for the build. Accepts a comma separated string of a tool chain or tool with option version. The tools are loaded in the order of right to left so any settings in the first tools override any tool defined after it.

Arguments

Must be a string defining a validate toolchain or tool file. Optional `<version>` can be added. More than on chain or tool can be loaded is they are separated by a comma. On some systems this might require adding a `"` to prevent the shell from breaking on the argument. For example, in PowerShell, saying `--tc=icl,mstools` has to be done as `--tc="icl,mstools"`

Examples:

```
Scons all --tool-chain=cl
```

Setup the latest version of the Microsoft Compiler tool chain found on the system.

```
Scons all --tool-chain=gcc_4
```

Setup the latest 4.x version of gcc install on the system. If the system has more than one version 4 compiler install, such as 4.2 and 4.4 the build system will select the 4.4 version.

```
Scons all --tool-chain=icl_12.1,mstools_9
```

Some tools adds value to a given tool chain, such as in this case which setup the Intel C/C++ Compiler version 12.1 to use Microsoft VS 2008 tool chain.

```
Scons all --tc=icl_12.1,gnutools
```

In this case which setup the Intel C/C++ Compiler version 12.1 to use the latest version of the GNU tool chain installed on the system. On most POSIX systems there is only one compiler and bin utility version installed. So this is often the default version.

```
--build-config=BUILD_CONFIG, --bcfg=BUILD_CONFIG, --cfg=BUILD_CONFIG
```

The configuration set to apply to the build. The default value is *debug*. Parts provides two primary configuration or *debug* and *release*. The debug configuration tend to not add optimization flags and add values to help with debugging, while the release configuration tends to add optimization flags while not adding flags to help with debugging. User can define their own configuration based on existing one, or completely independent values. Not all tools have configuration defined for them

Arguments

Must be a string defining a validate Configuration known by the build system.

Examples:

```
Scons all --cfg=debug
```

This is the same as saying “*Scons all*”. Build via applying debug flags.

```
Scons all --cfg=release
```

Build via applying release configuration flags.

```
Scons all --cfg=my_super_cfg
```

Build via applying my_super_cfg configuration flags.

--mode=MODE

Define a default set of modes that may affect how some Parts builds.

Arguments

A comma separate string of values

Examples:

```
Scons all --mode=use_logging,build_driver2
```

VCS Options:

--vcs-update=UPDATE, --update=UPDATE

Tell is we should update the any Parts that define a VCS object.

Arguments

If no value is provided the system will update all known Parts. Same as using the value of “true”, while a value of “false” can be used to force no updating checks happen at all. A comma separated list of part references can be used to update a select amount of Parts.

Examples:

```
Scons all --update=false
```

Do not update anything. Parts will not do skip all checks for being out of data.

```
Scons all --update
```

Update all components.

```
Scons all --update=mycomponent
```

Update the parts with name of mycomponent.

```
Scons all --update=comp1,comp3
```

Update the parts with name of comp1 and comp3. Comp2 will not be updated.

`--enable-vcs-clean, --vcs-clean`

For any component that is updated, make sure the update is clean. This removes all “unknown” files and revert any changes. This is a go back to “factory” settings option.

Arguments

No arguments required. Optional “true” and “false” can be given.

Examples:

```
Scons all --vcs-clean --update
```

Gets any checked in changes and removes any local differences. Should result in a clean setup as if you had done a fresh checkout.

`--enable-vcs-retry=VCS_RETRY, --vcs-retry=VCS_RETRY`

Controls if Parts will try to recover from some checkout or update problem via deleting and rechecking out the code. Often difference VCS tools can get in to odd states if there was an unexpected interruption. These often require trying to run some command like “clean” or manually deleting and rechecking out. This option is very useful in automate systems which can get in to a bad state when checking out code and would normally require some manual interventions to correct the problem. The system will retry only once. If it was unable to successfully retry to get the code the second time the system will fail with an error message.

Arguments

No arguments required. Optional “true” and “false” can be given.

Examples:

```
Scons all --vcs-retry
```

Any component that needs to be update because it points to a different branch will update and retry to update if there is some odd update issues preventing the update from success.

`--vcs-logic=VCS_LOGIC`

Control logic of how Parts will check that the VCS branch/repository has change to a different location or if the code even exists on disk.

Arguments

Value can be (ordered in fastest/least correct to slowest/most correct):

- none – Don't do any checks
- exists – Only checks if the Parts files exists
- check – check the cache file to see if what was stored from last run is different
- force – call vcs tool to see if what is on disk is what is defined in the Sconstruct file.

The more correct the test, generally the longer it takes to get the information. The default is *check* as this is general the best mix of speed verse correctness.

Examples:

```
Scons all --vcs-logic=force
```

Make sure any manual relocation on disk are correctly pointing to what is defined in the Sconstruct file

```
Scons all --vcs-logic=none
```

Skip all checks because of manual hacking, or to get a little extra startup speed.

`--vcs-job=VCS_JOBS, --vcsj=VCS_JOBS, --vj=VCS_JOBS`

Set the level of concurrent vcs jobs that can run a given time. Can help decrease the time needed to update or get all the sources in large projects or with servers that have high latency.

Arguments

Must have an integer value of 1 or greater. Value defaults to the value of the `-j` or `-jobs` option.

Examples:

```
Scons all --vj=5
```

Check out up to five items at once.

`--vcs-policy=VCS_POLICY`

Policy in how Parts should react if the automatic vcs check say a given component is out of sync with the value provided in the Parts call or does not exist on disk.

Arguments

Value can be:

- warning – print a warning that the component is out of sync and continue
- error – Print an error message and exit.
- message_update – print a message and update the component
- update – update the component without a message.

Examples:

```
Scons all --vcs-policy=error
```

Prevent building if the components are not checkout or the same as defined in the SConstruct. This would require that the user adds a `--update` options to get the sources on disk. This might be used to make sure as different version branches get updated in the main SConstruct file, the user as to make sure the new branches are updated explicitly before the code will be built.

General Options:

`--use-parts-site=USE_PART_SITE`

User provided part-site path. Overrides all the default location normally used.

Arguments

Requires path to location to use as part-site location

Examples:

```
Scons all --use-parts-site=c:\custom-site
```

Loads data in the custom-site location

`--disable-global-parts-site`

Disable Parts from using the part-site location in the system or user areas. This allow for cases when a default set of part-site overrides maybe installed that would prevent you code from building, because of using incompatible version or using values you don't want to have applied.

Arguments

No arguments accepted

Examples:

```
Scons all --disable-global-parts-site
```

Prevents incompatible or unwanted part-site settings from taking effect on the current build.

--disable-parts-cache

Disable Parts data cache from being used

Arguments

No arguments accepted

Examples:

```
Scons all --disable-parts-cache
```

Disable the cache from being used to speed up the build

--load-logic=LOAD_LOGIC, --ll=LOAD_LOGIC

The logic to use when loading the parts files for improved start speed of the build. The logic followed is to "all" when there is no .part-cache data existing or if the given case being built is not defined in the cache yet. If the cache exists then the default load logic or provided case is used.

Arguments

Current options are:

- all – the behavior of SCons by default is to load everything. Most correct but slowest.
- target – load all part files needed to build target even if they are up-to-date.

- min or default – try to load only Parts files that are out of date. Should be as correct as the **all** case, but a lot faster. This is the default load logic used.
- unsafe – this is basically a no depends load of the part files defined by the target. This case will load only the Part files directly mapped to the target and assume all dependent Parts are up-to-date. This is can be very fast, but can make bad bits.

Examples:

```
Scons foo --load-logic=all
```

Load everything from scratch.

```
Scons mypart:: --load-logic=unsafe
```

Load the part named **myparts** and an sub-part defined by this Part and read it, assuming all dependent parts are up-to-date.

```
Scons mypart.sub1 --ll=unsafe
```

Load the part named **myparts.sub1** assuming all dependent parts are up-to-date. Note the this case will load the parent part, but any other sub-part (ie sub2, sub3) will not be loaded.

```
--ccopy=CCOPY_LOGIC, --ccopy-logic=CCOPY_LOGIC, --copy-logic=CCOPY_LOGIC
```

Control how Parts copy logic will work must be when it copies files around. Logic allows the creation of links instead of full copies, and allows for fallbacks when certain cases can be done because of moving data between different locations that don't allow a link of some kind to be created. Using links can save space and decreases build times.

Note on Microsoft Windows systems! Symbolic link option will work only on Vista and newer systems using NTFS. You may have to set a group policy setting to give yourself permission to create then as by default only administrator have rights by default. Hard links should work on any NT based system using NTFS.

Arguments

Current options are:

- hard-soft-copy – current default, will try to make a hard link, then a symbolic link then do full copy
- soft-hard-copy -- will try to make a symbolic link, then a hard link then do full copy
- soft-copy – will try to make a symbolic link then do full copy
- hard-copy – will try to make a hard link, then do full copy
- copy – do full copy only

Examples:

```
Scons foo --ccopy-logic=copy
```

Always do full copies

```
Scons foo --ccopy-logic=hard-copy
```

Try to do a symbolic link, if that fails do a full copy

--disable-section-suppression

Disable process suppression of any sections defined in a Part via the SUPPRESS_SECTION variable. This allows for cases in which a section may have been disabled because it does not build. For example you might disable a unit tests in a Part via saying:

```
Part('foo.part',SUPPRESS_SECTION=['UTEST'])
```

This would might be done because it just does not work and till it is fixed you don't want to prevent the standard **utest::** target from working. Tagging the part this way makes it clear that something is broken and needs to be fixed. The use of this option on the command line allow the developer of foo to disable the suppression of their unit tests so they can correct want is broken.

Arguments

No arguments accepted

Examples:

```
Scons utest::foo --disable-section-suppression
```

Build the unit tests for foo by disabling requested suppression of the utest section in the Part call.

--cfg-file=CFG_FILE, --config-file=CFG_FILE

User defined configuration file used to store common settings. Parts will load parts.cfg by default.

Arguments

Path with configuration file must be provided.

Examples:

```
Scons all --cfg-file=custom_value.cfg
```

Use values defined in custom_value.cfg file.

Variables

Parts defines a number of variables to control different aspects of the build, such as directories used for building, or install, to control certain behaviors. Some variable have command line counterparts and are provides to help with configurability of the system.

Packaging

PACKAGE_GROUP_FILTER

Defines a dictionary of package groups names with callable objects to be used for sorting nodes to into different package groups. It suggested that using the packaging APIs `ReplacePackageGroupCriteria(...)`, `AppendPackageGroupCriteria(...)`, `ReplacePackageGroupCriteria(...)` to set values here instead.

Install

Defines location for where files are installed with the Install API. Some values such as help and documentation may not be well defined. In these cases defining them to the same location maybe helpful. A few variables control certain behaviors.

INSTALL_ROOT

Define the root directory for the install Sandbox. Most of the variable of the form `INSTALL_<VALUE>` install under this directory in the form of `${INSTALL_ROOT}/<VALUE>`. This valuable should define a root structure that should allow correct side by side building of different build configurations.

INSTALL_LIB

Defines the location for installing shared libraries

INSTALL_BIN

Defines the location for installing binaries, or programs to run

INSTALL_API

Defines the root location for installing API related files, such as static libraries or header files. Finer control can be achieve with the use of `sub_dir` argument on the `InstallSample()` API call.

INSTALL_INCLUDE

Defines the location for installing include files.

INSTALL_CONFIG

Defines the location for installing configuration files

INSTALL_DOC

Defines the location for installing documentation

INSTALL_HELP

Defines the location for installing for installing help files

INSTALL_MANPAGE

Defines the location for installing manpages

INSTALL_MESSAGE

Defines the location for installing message catalogs

INSTALL_RESOURCE

Defines the location for installing resources

INSTALL_SAMPLE

Defines the location for installing samples, much like `INSTALL_API` this is only a root directory. Finer control can be achieved with the use of `sub_dir` argument on the `InstallSample()` API call.

INSTALL_DATA

Defines the location for installing general data files

INSTALL_TOP_LEVEL

Defines the location for installing top level files that should be installed in the root location, such file might be a `readme.txt` or license information

PKG_NO_INSTALL

Defines the location for installing items that should not be installed, but maybe needed to create the final package, or are useful for different reasons.

INSTALL_PYTHON

Defines the location for installing python based files

INSTALL_SCRIPT

Defines the location for installing general scripts

AUTO_TAG_ON_INSTALL

Defines if installed nodes should be automatically meta-tagged with values in the AUTO_TAG_INSTALL variable, by default this value is True.

AUTO_TAG_INSTALL

Defines a list of tuples, where the first item in the tuple is a string that defines a simple wild card pattern, such as ***.pdb**. The second item is dictionary key values to be assigned the to node, where the key is the name of meta value to be used and the value is the to assign.

Example:

Make sure all PDB file are installed in the sandbox but are not included in the package.

```
SetOptionDefault (AUTO_TAG_INSTALL=[ ('*.pdb', {'no_package': True}) ])
```

INSTALL_LIB_PATTERN

This is the file pattern used by InstallTarget to determine if a file should be installed in the INSTALL_LIB directory. Currently the default is ['*.so', '*.lib']

INSTALL_BIN_PATTERN

This is the file pattern used by InstallTarget to determine if a file should go into bin. For windows based platforms the pattern is ['*.dll', '*.DLL', '*.exe', '*.EXE', '*.com', '*.COM']. For POSIX based system it is ['*']. In the POSIX case the INSTALL_BIN_PATTERN is checked last. This means it only matches if the given file is not anything else.

Sdk

Defines location for where files are copied with the SDK API. Some values such as help and documentation may not be well defined. In these cases defining them to the same location maybe helpful. A few variables control certain behaviors.

SDK_ROOT

Define the root directory for the for the SDK area. Most of the variable of the form SDK_<VALUE> install under this directory in the form of \${SDK_ROOT}/<VALUE>. This valuable should define a root structure that should allow

correct side by side building of different build configurations. The SDK variable are defined by default to lay out a component based SDK. This as an effect of longer paths to be added to components that depend on lots of different items. The SDK directory can be setup to more product based, where all components share a common set of directories. In this case the only a few directories are added.

SDK_LIB

Defines the location to copy libraries, such as shared and static libraries

SDK_BIN

Defines the location to copy programs and other binaries

SDK_INCLUDE

Defines the location to copy include related files

SDK_API

Defines the location to copy API related files

SDK_CONFIG

Defines the location to copy configuration related files

SDK_DOC

Defines the location to copy documentation

SDK_HELP

Defines the location to copy help files

SDK_MANPAGE

Defines the location to copy manpages files

SDK_DATA

Defines the location to copy general data files

SDK_MESSAGE

Defines the location to copy message catalog file

SDK_RESOURCE

Defines the location to copy resources file

SDK_SAMPLE

Defines the location to copy sample files

SDK_TOP_LEVEL

Defines the location to copy top level related files related to the install

SDK_NO_PKG

Defines the location to copy files that should not be packages, but need to be installed

SDK_PYTHON

Defines the location to copy python related files

SDK_SCRIPT

Defines the location to copy general scripts

USE_SRC_DIR

This value controls globally if the SDK directories are using for path that are passed as dependences. This is useful when debugging a program as the debugging information points to the original source location, which helps in debug, fix, compile, and repeat development cycles. Defaults to False, value does not override values in SDK functions if they are set to True.

CREATE_SDK

This value controls if a SDK is made at all. If set to False, this will have the same effect of setting USE_SRC_DIR to True as well. Default to True. Helpful in debugging or speeding up the build if SDK are not needed, however some Parts may setup their include paths to be different from how the source is layout in the original source directory. This can lead to compiling issues.

SDK_LIB_PATTERN

This value defines the pattern of files to treat as library files on windows or POSIX system via the SdkTarget method. Currently defined as ['*.lib', '*.LIB', '*.a', '*.A', '*.so', '*.sl'])

SDK_BIN_PATTERN

This value defines the pattern of files to treat as programs via the `SdkTarget` method. On win32 system it is defined as ['*.dll', '*.DLL', '*.exe', '*.EXE', '*.com', '*.COM', '*.pdb', '*.PDB'] while on Posix systems it is ['*'], or if the file is not a `SDK_LIB_PATTERN`, it is a bin file. Since programs on Posix systems don't use a file extension, there is no other known way to test via a string match if the file is a program or not.

Building

BUILD_DIR_ROOT

Defines the root path where all build action will happen under. The path by default tries to lay out a structure to prevent build conflicts for different build setups.

BUILD_DIR

Defines the path where an item for a given Part will build.

CONFIG

Defines the value of the configuration to be used for building. This maps to the `--build-config` command line option. The command line option should be the preferred way to set the value on the command line. This value should be used to set the configuration with `SetDefaultOption()`. Because configuration can be defined with inheritance in Parts, it is recommended that the `isConfigBasedOn()` is used for testing if a certain configuration is being used.

duplicate_build

Controls if the SCons should duplicate files in the build area, by default this is off.

toolchain

Defines a string that defines what tool chain to be used, that matches the value used with the command line option `--tool-chain`.

TARGET_PLATFORM

Defines what platform we are building for. This can be set with the `--target-platform` command line option.

HOST_PLATFORM

This is a read only value that defines the current platform we are building on.

SUPPRESS_SECTION

This value can be passed to a Part as a list of string value to tell Parts to not process a given section type. Currently this only makes sense for "utest".

Example:

```
Parts("foo.parts", SUPPRESS_SECTION = ['utest'])
```

Would prevent the processing of any `utest()` calls in the Parts and sub-parts. This is useful if there is a component that for some reason is not working correctly, such as the test doesn't build, to turn off this component in a way to not prevent the whole product from working. This also provides a nice way to mark which components need to be improved in the Sconstruct file. Adding the value of `--verbose=warning` will provide warning message for any component that has suppressed sections in it.

The use of **--disable-section-suppression** will allow for a user a easy way to prevent suppression to allow the user to correct any issues without needing to modify the Sconstruct file. This currently is a global value, future drops may allow for per component level control.

CONFIGURED_TOOLS

Defines what tools Parts will try to load configuration value for. This value is set internally via the `ToolChain()` api.

AUTO_RPATH

Defines if Parts should try to define RPATH value for platforms that support it, default value is True

MAPPER_BAD_ALIAS_AS_WARNING

Controls when mapping dependency information, if a failed mapping results in a warning message or an error message with exit. The default value is true. This will change to False in future drops.

Unit testing

BUILD_UTILITY_CONCEPT

This variable defines the Alias prefix used to define the concept of building the unit tests.

RUN_UTILITY_CONCEPT

This variable defines the Alias used to define the concept of running the unit tests after it is built.

UNIT_TEST_ROOT

This is the root directory from the unit test sandbox.

UNIT_TEST_DIR

This is the directory use as the sandbox for a given test.

UNIT_TEST_ENV

This is the variable which defines extra value to be added to the shell environment when the test runs. Very useful for reproducible environment setup on different systems, instead of forcing the user with manual setup to have the test run correctly.

UNIT_TEST_TARGET

This is the value passed to the target argument of the UnitTest method.

UNIT_TEST_TARGET_NAME

This is the name of the final unit test executable. The name by default it defined to try to avoid overlapping of tests in different Parts with similar names, to avoid the “two environments define the same output” error. The default value is '`${PART_NAME}-${UNIT_TEST_TARGET}_${PART_VERSION}`'

UNIT_TEST_RUN_SCRIPT_COMMAND

This is the command that is run when the 'run_ustest' build target is used. By default this will try to run the program directly inside the unit test sandbox area via a default command of '`cd ${NORMPATH("$UNIT_TEST_DIR")} && ${RELPATH("INSTALL_BIN","UNIT_TEST_DIR")} ${UNIT_TEST_TARGET_NAME}`'. This can be redefine to run a custom script.

UNIT_TEST_RUN_COMMAND

This is the command, the generated script will run when it is called. Overriding this value allows the user to control what action the script will run. This is useful in cases of adapting existing unit testing system scripts, or to define custom behavior of the unit test system.

Part Variables

PART_NAME

The full name of the current Part. If PartName was not set the value returns the ALIAS (or PART_ALIAS). This value is what env.PartName() will return

PART_SHORT_NAME

The short name of the Part if defined or else returns the SHORT_ALIAS. This value is what env.PartShortName() returns.

PART_ROOT_NAME

The full name of the defined top level root Part. If the root name was not defined then the ROOT_ALIAS is used. If the current Part is the root, the name of the current Part is returned.

PART_PARENT_NAME

The full name of the parent Part if it is defined. If it is not defined, the PART_PARENT_ALIAS (or PARENT_ALIAS) is used. If there is no parent an empty string is returned.

PART_ALIAS or ALIAS

The full Alias value that is defined as \${PARENT_ALIAS} + '.' + \${SHORT_ALIAS}

ROOT_ALIAS or PART_ROOT_ALIAS

The top level alias of the defined top level Part. If this Part is the root Part the value return will be equal to the value of the ALIAS

PARENT_ALIAS or PART_PARENT_ALIAS

The parent Alias value if one exists, else returns a empty string

SHORT_ALIAS

The value defined in the Alias call of the Part function before it is appended to any parent Part alias value.

PART_VERSION

The value of the Part's version as a string. If the version is 1.23.4 this would return a string "1.23.4"

PART_SHORT_VERSION

The value of the Part's short version as a string. The short version is defined as A.B. or the first two values of the version. If the version is 1.23.4 the short version would be returned as a string of "1.23"

PART_MAJOR_VERSION

This is defined for convenience; it is defined as the first value of the version number.

PART_MINOR_VERSION

This is defined for convenience; it is defined as the second value of the version number.

PART_ROOT_SIG

This defines the signature value for the root Part environment that makes it different from the default environment. This is used to help provide unique paths or values that are reproducible.

PART_LOG_MAPPER

Defines the object being used to log data for a given Part

PART_DIR or SRC_DIR

This defines the directory in which the Part files is defined in

Logging

LOGGER

Set by the --log command line option. It is set to the object being used to log all output. Can be set with a user defined logging object.

LOG_ROOT_DIR

Defines the root directory to be used for outputting log files

LOG_DIR

Define the full path used for outputting log files for a given run.

LOG_FILE_NAME

Defines the name of the log file to be used, default the value is all.log. Certain logger may try to extend the name with a custom extension.

VCS

VCS

This defines an object that defines a number information about the VCS object being used for a given Part. Values can change different between VCS objects. Some common value are:

- TYPE - a string that defines the type of VCS used
- CHECKOUT_DIR - the full path to be used for checking out a given component

Some special value that are defined

VCS Svn

- REVISION - a string defining the revision that is checked out
- MODIFIED - a value telling is the checkout code is modified
- PARTIAL - value telling if what is checked out is a partial checkout
- TOOL - the SVN program being used to checkout
- SERVER_PATH - the full path being used to get the data from
- FLAGS - the flags being added as part of a SVN action.

UPDATE_ALL

Normally set by the `--update` command line options. This control updating files from any defined version control system.

CHECK_OUT_ROOT

The root directory in which file will be checked out under.

CHECK_OUT_DIR

The full path to check out a component to, this defined as the value of `VCS_DIR`, as this allow any given VCS type to define a special directory.

VCS_DIR

Defines a directory to checkout to via forwarding the value of `$VCS.CHECK_OUT_DIR`

VCS_FILESYSTEM_DIR

The full path to use when a file system VCS object is used

VCS_SVN_DIR

The full path to use when a SVN VCS object is used

FILE_SYSTEM_SERVER

The server or location to use for getting files over the network

SVN_SERVER

The SVN server to use for checkout data

SVN_FLAGS

Any special flags to use when get data form with SVN. The default value is --non-interactive

Miscellaneous

CCOPY_LOGIC

Defines the logic to be used for coping files with CCopy() function. The value is normally set via the command line option --ccopy. Values can be hard-soft-copy, soft-hard-copy, hard-copy, soft-copy,copy.

Extending Parts via Parts-site

Parts provides a number of ways to be extended to add new support for different feature, tools or to customize the behavior. This is done via use the of the parts-site directory. This is much like Parts version of scon-site but with differences. This area is where the users can add their own tool chains, configurations, tools and pieces to be used by Parts.

Locations

Parts will look for a parts-site directory in the same directory as the SConstruct and then in the users home directory, in a system location and finally under the parts install area. The directory can be named **parts-site** or **.part-site**. The form without the preceding dot will be looked in first as it the dot form is classically hidden on POSIX based system. User of Windows will also have a harder time creating this directory via the explorer interface as Windows tries to prevent users from making directories that start with a dot character, using the command line will be much easier. A complete list of directories that Parts will look in:

- <SConstruct directory>/parts-site
- SConstruct directory/.parts-site
- <home directory>/parts-site
- <home directory>/parts-site
 - On window this include the home directory and then the location of %APPDATA%
- <system directory>/parts-site
- <system directory>/parts-site
 - Where the system directory is:
 - Windows %ALLUSERSPROFILE%
 - Posix /usr/share/parts
 - Darwin /Library/Application Support/parts
- <Parts install directory>/parts-site

Defining a piece

One of the directories under parts-site is called pieces. File in this location are automatically loaded. They are like tools in SCons, except there is no special API such as generate() to have defined. You can generally do about anything here. It is suggested that it is best to define custom variable, function, or builders that you want to have shared. Many of Parts own API are define as a piece.

<fill in more details>

```
env.DependsOn ([Component ( 'A ', requires=REQ.HEADERS) ] )
```

Defining your own Tool Chain

This logic is still evolving. However the current form of it will only be added to. A tool chain is a set of tools and values to setup a given tool that the user wants to use for a given build run. These toolchains can be specified on the command line, config file, or in the SConstruct. The user should be able to make a strict tool chain that only

uses an exact set of tools or a flexible one that allows for auto detection of the best set, much like the current default tool logic in SCons. These files can be added to parts-site/toolchain/ directory. Any value found here will override any built in ones. All tools must define a function

```
def resolve(env, version) :
```

This function will take the cloned env and a string for the version to select. In reality, the version variable can be any value, however Parts uses the convention of passing a string of some number value such as 8.0 or 3.4.6. If no value was provided it will be None. This value can be used to help figure out what values to setup. The environment can be used to test for the existence of certain tools. This function needs to return a list of tuples. The value of the tuple is important as it will tell Parts if it should load another tool chain or tool when the Environment is configured. Parts will expect these forms:

- ("toolchain name",None)
- ("toolchain name","version")
- ("tool name",{ })
- ("tool name",functor)

The first two forms are easy to understand. It is the name of another tool chain and possible version value. The last two forms allow the user to pass data to help control the setup of a SCons tool. One form is a dictionary of key values that will be added during environment Construction to control the setup. The second allows for a call back function or object that will be passed an env to fill with values.

Examples:

Set up a Microsoft tool chain

```
def cl_setup(env,ver):  
    env['MSVC_VERSION']=ver  
    env['MSVS_VERSION']=ver  
    env['MSVC_INSTALL_ROOT']=None  
  
def resolve(env,version):  
    func=lambda x : cl_setup(x,version)  
    return [  
        ('cl',func),  
        ('mslink',func),  
        ('masm',func),  
        ('mslib',func),  
        ('midl',func)  
    ]
```

Set up a Microsoft tool chain (using dictionary)

```
dict={'MSVC_VERSION':ver,
      'MSVS_VERSION':ver,
      'MSVC_INSTALL_ROOT':None}

def resolve(env,version):
    dict={'MSVC_VERSION':ver,
          'MSVS_VERSION':ver,
          'MSVC_INSTALL_ROOT':None}

    func=lambda x : cl_setup(x,version)

    return [
        ('cl',dict),
        ('mslink',dict),
        ('masm',dict),
        ('mslib',dict),
        ('midl',dict)
    ]
```

Simple auto selection example

```
def resolve(env,version):

    host=env['HOST_PLATFORM']

    if host.OS=='win32':

        return [

            ('cl',None),

            ('mslink',None),

            ('masm',None),

            ('mslib',None),

            ('midl',None)

        ]

    elif host.OS=='posix':

        return [

            ('gcc',None),

            ('g++',None),

            ('gas',None),

            ('gnulink',None),

            ('ar',None)

        ]

    elif host.OS=='darwin':

        return [

            ('gcc',None),

            ('g++',None),

            ('gas',None),

            ('gnulink',None),

            ('ar',None)

        ]

    else:
```

```
print "Defaulting to Scons' default lookup"

return [('default', {})]
```

Defining your own Configuration

A configuration is a named set of settings and flags to be used by the tools when the code is building. Common configurations are debug and release. Parts ships with a basic debug and release configuration for most tools that does the basic settings needed to add debugging information and turning off optimizations for debug, and turning on optimizations and turning off debugging information for release. However these settings most likely will not meet your needs. To address this, custom configurations can be defined by the user. Parts also allows the user to create configurations that can be based on existing configurations. When being based on an existing configuration, the user can append, prepend, replace, filter any value that is needed, as well as, define a callback function to handle more complex setup logic.

Layout of configurations

Configurations are a set of files under a directory with the name “configuration”. To define a configuration one has to define:

1. An `__init__.py` file that calls `DefineConfiguration`, which will define the basic information of a configuration
2. A set of files, with a certain name format specifying the host and target platform combination, that allows settings to be defined for a given tool.
3. In each file define a set of settings and setup logic for different version ranges of a tool

The layout is designed to be easy to use and flexible.

Defining a configuration

The first step in defining your own configuration is to create a directory with the name of your desired configuration. For example we will define "debug". In this directory create a file with name `__init__.py`. Add a call to `DefineConfiguration`, for example:

```
from parts.config import DefineConfiguration

DefineConfiguration("debug", dependsOn='default')
```

`DefineConfiguration` function defines the name and it's relationship to another configuration in Parts. When a configuration depends on another configuration, that configuration will be loaded and used as a base set of flags on top of which your configuration will add and modify. Parts ships with a debug, release and default configuration. In Parts, default is the base of all configurations defined. However if you don't want your configuration to be based on one of these, you can pass `None`.

Define a file name

When Parts loads these settings it will look for a set of files with a certain name format, in a directory labeled with the name of the configuration. These names allow the user to define how the settings defined in the file should be loaded and treated. For example a file with the name of gcc.py will be used for any defined platform, while a file with the name of gcc_posix-x86_posix-x86.py would only be used when doing a build on a host system of posix-x86 and targeting the output for same system. More detailed names, such as gcc_posix-x86_posix-x86.py, are preferred over less detailed name, such as gcc.py. The current set of names and order of preference is below:

<tool name>_<HOST_OS>-<HOST_ARCH>_<TARGET_OS>-<TARGET_ARCH>.py

<tool name>_<HOST_OS>-<HOST_ARCH>_<TARGET_OS>-any.py

<tool name>_<HOST_OS>-any_<TARGET_OS>-<TARGET_ARCH>.py

<tool name>_<HOST_OS>-any_<TARGET_OS>-any.py

<tool name>_<HOST_OS>-<HOST_ARCH>_any-<TARGET_ARCH>.py

<tool name>_<HOST_OS>-<HOST_ARCH>_any-any.py

<tool name>_<HOST_OS>-<HOST_ARCH>_any.py

<tool name>_<HOST_OS>-<HOST_ARCH>.py

<tool name>_<HOST_OS>-any_any-<TARGET_ARCH>.py

<tool name>_<HOST_OS>-any_any-any.py

<tool name>_<HOST_OS>-any_any.py

<tool name>_<HOST_OS>-any.py

<tool name>_any-<HOST_ARCH>_<TARGET_OS>-<TARGET_ARCH>.py

<tool name>_any-<HOST_ARCH>_<TARGET_OS>-any.py

<tool name>_any-any_<TARGET_OS>-<TARGET_ARCH>.py

<tool name>_any_<TARGET_OS>-<TARGET_ARCH>.py

<tool name>_any-any_<TARGET_OS>-any.py

<tool name>_any_<TARGET_OS>-any.py

<tool name>_any-<HOST_ARCH>_any-<TARGET_ARCH>.py

<tool name>_any-<HOST_ARCH>_any-any.py

<tool name>_any-<HOST_ARCH>_any.py

<tool name>_any-<HOST_ARCH>.py

<tool name>_any-any_any-<TARGET_ARCH>.py

<tool name>_any_any-<TARGET_ARCH>.py

<tool name>_any-any_any-any.py

```
<tool name>_any_any.py  
  
<tool name>_any.py  
  
<tool name>.py
```

The goal of this look up is to allow the user to easily define settings at the required level of detail based on the need of the tool and platform. This allows the user to easily define settings for any generic platform while still allowing the ability to specialize for a more specific platform.

Define settings for a tool

To define settings, each file needs to define a configuration object with the name of config, such as:

```
config=configuration(map_default_version)
```

The configuration object takes two arguments:

`configuration(default_ver_func,post_process_func=None)`

- **default_ver_func** -- This value is a function that takes an env argument. It has to return the version value of the tool that was setup in the tool chain.
- **post_process_func** -- This optional function takes an env argument. It will be called after all the defined settings for all tools have been applied. This is useful for tools that might need to add values based on certain settings of values or versions of other tools that had been defined. For example the Intel compiler will want to define compatibility flags based on version of the Microsoft or GNU compiler being used.

Once the config object is defined, call the method *VersionRange* to define what settings you want to have applied. The definition of this method is:

`VersionRange(ver_rng,replace={},filter={},append={},prepend={},prepend_env={},append_env={},post_process_func=None)`

- **ver_rng** -- a string defining the version range that these settings apply to. The string follows the same rules as the VersionRange object.
- **replace** -- the settings that we want to have replaced from the base configuration. Any value here will overwrite any previously set value. If defined, it is applied first.
- **filter** -- the key values will be searched for and removed if they exist in the environment. This is useful for removing a certain set of flags, such as removing all warning flags and ensuring that only a specific flag is active. If defined, it is applied second.
- **append** -- these key values are added to the end of the list of any existing value. This is done with SCons AppendUnique logic
- **prepend** -- these keys values are added at the start of the list of any existing value. This is done with SCons PrependUnique logic
- **prepend_env** -- the values are added to the Environment that will run this tool. The given value will be added to beginning of any existing value
- **append_env** -- the values are added to the Environment that will run this tool. The given value will be appended to any existing value

- `post_process_func` -- the same as the `post_process_func` defined in configuration, except that it will only be called in case of a tool with a matching version range being loaded

This function should allow about any set of values to be applied and configured. To help with defining the dictionaries of values that might be applied a function `ConfigValues` has been added to define sets of values in a prettier way. This function takes `**kw` as an argument. In python this is a generic variant argument list.

A simple example of a possible debug definition for cl debug might look like this:

```
def map_default_version(env):
    return env['MSVC_VERSION']

config=configuration(map_default_version)

config.VersionRange("6.0",
                    append=ConfigValues(
                        CCFLAGS=['/Od', '/MDd', '/W3', '/GZ'],
                        CXXFLAGS=['/EHsc', '/GR']
                    )
)

config.VersionRange("7-*",
                    append=ConfigValues(
                        CCFLAGS=['/Od', '/MDd', '/W3', '/Zc:wchar_t', '/RTC1'],
                        CXXFLAGS=['/EHsc', '/GR']
                    )
)
```

In this simple example we simply apply values to any existing values. For version 6 we apply a set of values, for version 7 and newer we define a different set of flags as these versions changed the compiler switches.

Simple example of Configuration

One might want to have their configuration be based on the default debug configuration that is shipped. In this simple example we will create a `my_debug` configuration. First we need to create a `my_debug` directory under an

location that will be looked in. We will use the home directory parts-site/configuration directory for this. Create a ~/parts-site/configuration/my_debug directory. (For Windows users this is c:\users\<user-id>\parts-site\... for Vista and newer, or c:\Document and Settings\<user-id>\parts-site\... for XP user)

Next we need to create a __init__.py file in this directory. Add these contents in it:

```
from parts.config import DefineConfiguration

DefineConfiguration("my_debug", dependsOn='debug')
```

This will define the relationship of this configuration to Parts. Now Parts will load the setting “debug” before it applies your modification.

Next create a file that we will want to store our settings in. For this example we will apply settings to the CL compiler so we will make a file named **cl.py**. In this file we will modify the settings with some new values for the 8.0 compiler only. For example, we want to have /W1 flags be used, and any other warning flags removed.

```
def map_default_version(env):

    return env['MSVC_VERSION']

config=configuration(map_default_version)

config.VersionRange("8.0",

                    filter= ConfigValues(CCFLAGS=['/W2', '/W3', '/W4'])

                    append=ConfigValues(CCFLAGS=['/W1'])

                    )
```

Now when the code is build with the cl compiler version 8 we will have used the base debug settings but would have removed the all warning flags except '/W1' and added /W1 if it was not already added.

Adding Tools with Parts

SCons has the ability to be extended by new tools by the user. The tool can do many things in SCons. For the subject of this discussion it will primarily be a file that sets up a builder with a control variable and a shell environment to run some command line tool.

Basic requirements

All tools in SCons are required to contain two functions:

- exists(env) -- returns true if the tools can be configured based on values that control tool setting defined in the env variable.
- generate(env) -- configures the environment with settings need to call and run the tool builder in the supplied environment

Problems

This interface is very simple and in practice works well, except that:

1. There is no standard for what a given tool file defines to control its setup. The selection logic for a version was never the same between tools. It also was difficult as some tools would allow the user to select different versions of the tool on a platform, and some would not.
2. The logic to look up a tool is often similar but was never shared. This required the users to copy and paste the same logic with small tweaks when creating a new tool.
3. The handling of what the tool did when a tool was found or not found was often different. Some tools would error with a message that the tool could not be configured while some would do nothing and wait for the build to fail later with a missing tool or some other odd message.
4. It is difficult to add support for new versions of a tool. When some tools released a new version it was almost impossible for the average user to add support for the new tool, without being an expert. Even in those cases it was hard, painful work to refactor the code to correctly handle the new and old logic.

Improved Requirements

Parts adds some new code and requirements to help standardize the tool setup logic and to reduce the difficulty of adding new tools or extending existing ones with new version or platform support.

First, all tools defined in Parts have the requirement to be configured by a new set of variables. The values allow any tool to easily configure itself based on version, script and target platform (ie cross builds). The values are:

- `<TOOLSET_NAME>_VERSION` -- when set to None or does not exist the tool will be configured to the latest found version. If set to a string value, that version will be configured.
- `<TOOLSET_NAME>_INSTALL_ROOT` -- when set to None or does not exist the tool will be configured based on standard install locations. Else it is a path that will be treated as the install root when configuring the tool, based on the provided `<TOOLSET_NAME>_VERSION`. Useful for custom install locations for tools on disk.
- `<TOOLSET_NAME>_SCRIPT` -- when set to False, standard settings are used to look for and setup the tools' shell environment. When True the tools will try to use preknown scripts location based on the `<TOOLSET_NAME>_VERSION` and `<TOOLSET_NAME>_INSTALL_ROOT` that setup the environment. If set to a string/File node, it will try to load the user script provided in the string.
- `TARGET_PLATFORM` -- this value is used to help the tool configure itself for a possible cross build. Not all tools support cross build.

In all cases if the tools can't be found on disk the Tool will error with a message that the tool was not found. It may provide information on what versions where found. In all cases if these values are provided they will control the tool setup. After the setup these values will be set to values used to setup the tool. This means

`<TOOLSET_NAME>_VERSION` will be set to the version of the tool configured and

`<TOOLSET_NAME>_INSTALL_ROOT` to the value of the install location used.

ToolSetting and ToolInfo

Since much of the logic to define this for any given tool is 99% the same most of the time Parts provides some objects to help with defining a tool lookup logic and setting up basic settings. These objects are broken into two basic objects.

- **ToolSettings** -- This object defines basic lookup and setup logic. It is a container of Tool Info objects. It provides means to register new information for new version, host and target platform combinations. It also provides a standard means to test for existence and to add values into the SCons environment when a tool is being configured
- **ToolInfo** -- this object holds information on where to look for the tool when being configured for a given version, host, target combination. It also contains values to be added to the shell environment needed to run the tools.

These objects handle the finding and setup of the values for the shell environment. It does not handle setup of builders as these tend to be custom. These objects can be derived from, if a tool platform has a slightly different standard look up logic, or a little customization might help a great deal in simplifying the look up. An example of this would be GNU tools, that for the most part, follow a common look up logic pattern that is different from most other tools.

Tool Settings object

This object provides the primary API for registering and setting a tool. It caches known values to help improve setup time, and to speed up the build

Declarations:

TOOLSETTINGS(NAME)

- **name** -- This is the name of the tool. Parts will make a namespace object in the Environment to allow easy access to common values such as version, install root and other information via SCons subst() call.

Methods:

EXISTS(ENV, TOOL=NONE, **KW):

Returns True if the tool that would be configured was found

- **env** -- the Environment that contains the values to control what tool version location we want to find.
- **tool** -- this is the name of the tool to test for existence. If None it will use the tool provided in the ToolInfo object.
- ****kw** -- to override configuration value used for the testing.

MERGESHELLENV(ENV):

- **env** -- the Environment to use for values to control how we configure the tool and setup the Environment.

REGISTER(HOSTS, TARGETS, INFO):

Register a list of information objects based on a set of host and target platforms

- **host** -- a list of SystemPlatform objects used to define the host platforms that can apply these tool settings.
- **target** -- a list of SystemPlatform objects used to define the target platforms for these tool settings.

Examples:

Ideally one will create one object for a given tool or a group of tools that exist together anyways. For example:

```
msvc=ToolSetting('MSVC')
```

This object would be used in the tools.py file to setup values. A simple example of how one might implement a tool's Exist() function would be:

Have ToolSettings use the default tool defined in the ToolInfo object that was registered to see if it exists on disk.

```
def exists(env):  
  
    return msvc.Exists(env) #msvc_exists(env)
```

or (in case of groups of tools, but we want to verify they are on disk)

```
def exists(env):  
  
    ret=False  
  
    if is_win64():  
        ret=msvc.Exists(env, 'ml64')  
  
    else:  
        ret=msvc.Exists(env, 'ml')  
  
    return ret
```

The generate() function will often look like:

```
def generate(env):  
  
    """Add Builders and construction variables for <your tool> to an  
Environment."""  
  
    #Setup builders  
  
    ...  
  
    #setup environment  
  
    msvc.MergeShellEnv(env)
```

In Parts, calls to register tool settings are most often defined in some file under a <name>Common directory. For example Intel tools are defined under a IntelCommon, Microsoft tools under MsCommon, and Gnu tool under GnuCommon. There is no strict rule here. This is done however to make it easier to decouple custom tool overrides from the built-in ones, as current SCons logic tends to bind tool files together. This helps a bit with separation, for

other tool extensions, add-ons or overrides. The Register function is the most important function, however it will be shown later after ToolInfo and Namespace objects are defined.

Tool Info Object

The tool info object is used to define information on how to find the tool and what to setup in the environment if the tool is found. It runs on the basic premise that most tools have a standard install directory or some possible script to setup the environment. This object will be tested by the ToolSetting object to see if the version it supports exists. It will look for a possible install root, or use the user override to create a path (via predefined area based on the install root or running a script) to test for the existence of the tool. This object caches values to decrease construction time of many environments.

Declarations:

TOOLSETTINGS(VERSION,INSTALL_SCANNER,SCRIPT,SUBST_VARS,SHELL_VARS,TEST_FILE)

- version – This is a string that defines the version value.
- install_scanner -- this is the list of finder objects, or a scanner object. If this is a list it will assume it is a list of finder objects that can be used to look at a location for information about the install root. The first one that returns a find is the value that is used for the install root. If it is a scanner object, the scanner will be queried to scan for all versions.
- script -- takes a ScriptFinder object that is provided information on where to find the script.
- subst_vars -- this is a dictionary of extra values that can be used as substitution variables.
- shell_vars -- this is a dictionary of values that will be added to the shell environment when the tool runs. PATH must be specified in dictionary as this value will be used to find the tool.
- test_file -- this is the name of the tool to test for.

Finders

Finders are a set of objects defined to provide information to the ToolInfo object. These objects provide a simple and clean way to check for the existence of a path on disk, in the registry or in the user Shell environment. These are objects so their logic to find the data is delayed until requested via an overloaded __call__ operator.

Declarations:

PATHFINDER (PATHS)

- paths – This is a list of paths to test if they exist. The list will be searched in the order first to last.

REGFINDER (KEYS,REL_PATH=NONE)

- keys – This is a list of registry keys to use. The list will be searched in the order first to last. The key's value will be used as a path and checked to see if it exists on disk.
- rel_path -- this value is added to the path we test for existence. It provides a way to modify the INSTALL_ROOT value if the path the registry points to is not the ideal install root.

ENVFINDER (KEYS,REL_PATH=NONE)

- `keys` – This is a list of shell environment variables to use. The list will be searched in the order first to last. The key's value will be used as a path and checked to see if it exists on disk.
- `rel_path` -- this value is added to the path we test for existence. It provides a way to modify the `INSTALL_ROOT` value if the path the environment points to is not the ideal install root.

ScriptFinder

The script finder is different from the other finders as it needs the install root to be known to find the script. The overloaded `__call__` function will be provided with a Environment with the ToolInfo `subst_vars` added to the tools namespace.

Declarations:

SCRIPTFINDER (NAME,ARGS=NONE)

- `name` – This is a string with the name of the script to find. Values will `subst()` before testing for existence.
- `args` -- this is a optional list of arguments that will be added when the script is called.

Namespace

The namespace is not an object that is directly created by the user. It is an object to make it easier to separate common values in the SCons environment. It allows for an easy to use `'.' <dot>` separator to be used when a value is be substituted in via SCons `subst` call. For example one can get the `INSTALL_ROOT` for the Intelc tool via:

```
env.subst("INTELC.INSTALL_ROOT")
```

or

```
env["INTELC"]["INSTALL_ROOT"]
```

or

```
env["INTELC"].INSTALL_ROOT
```

When a ToolInfo object adds the optional `subst_vars` values, they are added as a Parts Namespace object. Along with this information ToolInfo will also add the values of `INSTALL_ROOT`, `VERSION`, and `TOOL`.

Example of a Tool Definition:

This sample is a simplified version of the Microsoft compiler tool. In Parts the ToolSetting object is defined as this:

```
msvc=ToolSetting('MSVC')
```

In Parts a number of different files import the `msvc` ToolSetting object and make calls to register different versions of the Microsoft compiler. For example in `vc_old.py` the definitions of version 6 to 7.1 are defined. For version 6 of the compiler support would have the ToolSetting register call with these settings:

```

msvc.Register(

    hosts=[SystemPlatform('win32','any')],

    targets=[SystemPlatform('win32','x86')],

    info=[

        ToolInfo(

            version='6.0',

            install_scanner=[

                RegFinder([

                    r'Software\Wow6432Node\Microsoft\VisualStudio\6.0\Setup\Microsoft Visual
                    C++\ProductDir',

                    r'Software\Microsoft\Microsoft\VisualStudio\6.0\Setup\Microsoft Visual
                    C++\ProductDir'

                ]),

                EnvFinder([

                    'VS60COMNTOOLS'

                ],'../../VC98'),

                PathFinder([

                    r'C:\Program Files (x86)\Microsoft Visual
                    Studio/VC98',

                    r'C:\Program Files\Microsoft Visual Studio/VC98'

                ])

            ],

            script=ScriptFinder('${MSVC.VSINSTALL}/Common7/Tools/vcvars32.bat'),

            subst_vars={

                'VCINSTALL': '${MSVC.INSTALL_ROOT}',

                'VSINSTALL': '${MSVC.INSTALL_ROOT}/..'

            },

            shell_vars={

```

```

        'PATH':

            '${MSVC.VCINSTALL}/bin/'+os.pathsep+

            '${MSVC.VSINSTALL}/Common/msdev98/BIN/'+os.pathsep+

            '${MSVC.VSINSTALL}/Common/TOOLS/'+os.pathsep+

            '${MSVC.VSINSTALL}/Common/TOOLS/WINNT/'

            ,

        'INCLUDE':

            '${MSVC.VCINSTALL}/ATL/INCLUDE/'+os.pathsep+

            '${MSVC.VCINSTALL}/MFC/INCLUDE/'+os.pathsep+

            '${MSVC.VCINSTALL}/INCLUDE/'

            ,

        'LIB':

            '${MSVC.VCINSTALL}/MFC/LIB'+os.pathsep+

            '${MSVC.VCINSTALL}/lib/'

            ,

        'SYSTEMROOT':SCons.Platform.win32.get_system_root()

    },

    test_file='cl.exe'

)

]

)

```

First the arguments of:

```
hosts=[SystemPlatform('win32','any')],  
targets=[SystemPlatform('win32','x86')],
```

defines that this setup will support setting up the tool only on Windows platforms of any architecture (we assume and Intel based chipset), and can output only x86 based binaries. This means that a TARGET_PLATFORM set to win32-x86_64 will not find a version 6 compiler as we did not register a version 6 that supports native binary outputs for x86_64 chips. The host platform set to win32-any allows Parts to find version 6 on any system that has a TARGET_PLATFORM set to 'x86'. It should be noted that if there was a host platform of "win32-arm" then this tool would try to find the vc6 compiler. However vc6 can't run or build on ARM platforms. If we wanted to correct this by preventing these tool settings from being used to setup vc6 on an win32-arm platform we could define host as:

```
hosts=[SystemPlatform('win32','x86'), SystemPlatform('win32','x86_64')]
```

Doing this would allow vc6 to be found on architectures that can run 32-bit x86 code.

The last argument of info takes a list of ToolInfo objects. we could define all versions 6 through 7.1 in one register call, or we can define them in separate register calls. For readability of this example, we define only one version. The ToolInfo object defines the settings for version 6. This is clear from the first argument passed in. The second argument is a list of Finder objects that are ordered in the most preferred to least preferred way to locate in VC 6.

The first Finder in the list is a RegFinder that will look in the registry on windows. The order of the registry keys prefers finding certain keys over others. For example, on 64-bit Windows systems the keys for 32-bit programs go in a different location in the registry, We look for the 64-bit key first to prevent us from finding duplicate keys that exist on 32-bit and 64-bit systems, but where on the 64-bit system the duplicate key would not be correct. Another case of need for more than one look up location is when the sample tool is installed but with different package names. For example VC2008 has a "std" and "express" version. In the case of VC2008 the compiler for the express and std version are installed in the same location on disk, but to find this in the registry requires looking in different locations. In this case we don't need to care about whether this is the Express version or not, only if the 2008 compiler was installed.

The next Finder is an EnvFinder which looks in the user Shell Environment for a value that points to a location on disk. This value for VC6 points to the Common Tools area under the compiler install root. To redirect it to a better location that would be equal to the install root we would find in the RegFinder we pass it a rel_path value to offset the path to a better location. This finder would be used if and only if the RegFinder could not find the valid location in the registry.

The last finder is a PathFinder. It looks for the paths to exist on disk in known default install locations. This is the last fallback if nothing can be found as could happen if the compiler was manually installed by the user. If the user did do a manual install, and did not copy it in the standard install location, the user can provide a MSVC_INSTALL_ROOT value that points to the correct location. As long as the install layout is the one Microsoft would use everything should be found.

The next ToolInfo argument is script. It is passed a ScriptFinder object. Notice that it is passed a string value of '\${MSVC.VSINSTALL}/Common7/Tools/vcvars32.bat'. The value of MSVC is the namespace and VSINSTALL is a value we added in the next argument. This value will expand to the correct value to find the script.

The `subst_vars` argument allows for the addition of some special variables. These values can be used to make more organized logic or add special values that can be substituted later or found by the user to get information. In our example for `vc6` we have:

```
subst_vars={  
    'VCINSTALL': '${MSVC.INSTALL_ROOT}',  
    'VSINSTALL': '${MSVC.INSTALL_ROOT}/..'  
}
```

`VCINSTALL` expands to the location of `INSTALL_ROOT`. We can interchange the values of `MSVC.VCINSTALL` with `MSVC.INSTALL_ROOT`. `VSINSTALL` points to a different location needed to setup the environment. In the case of VC 2008 this value can be set with the framework SDK information such as:

```
subst_vars={  
    'VCINSTALL': '${MSVC.INSTALL_ROOT}',  
    'VSINSTALL': '${MSVC.INSTALL_ROOT}/..',  
    'FRAMEWORK_ROOT': framework_root(),  
    'FRAMEWORK_ROOT64': framework_root64()  
}
```

In this case we added some new values that can be used to create a path in a easy, reusable way for paths that need the framework SDK.

The `shell_vars` dictionary argument will be added to `env["ENV"]` after it is fully substituted. It is important that this has at least the `PATH` value set. The values for `INCLUDE`, `LIB` and others are set here. The use of the namespace object makes it easy to cleanly add substitute values in the strings without having to deal with global values that may not be set correctly.

```

shell_vars={

    'PATH': '${MSVC.VCINSTALL}/bin/' + os.pathsep +
    '${MSVC.VSINSTALL}/Common/msdev98/BIN/' + os.pathsep +
    '${MSVC.VSINSTALL}/Common/TOOLS/' + os.pathsep +
    '${MSVC.VSINSTALL}/Common/TOOLS/WINNT/',

    'INCLUDE': '${MSVC.VCINSTALL}/ATL/INCLUDE/' + os.pathsep +
    '${MSVC.VCINSTALL}/MFC/INCLUDE/' + os.pathsep +
    '${MSVC.VCINSTALL}/INCLUDE/',

    'LIB': '${MSVC.VCINSTALL}/MFC/LIB' + os.pathsep +
    '${MSVC.VCINSTALL}/lib/' ,

    'SYSTEMROOT': SCons.Platform.win32.get_system_root()

```

The last value is the name of a file to test for existence. This allow the code to confirm that this version exists. It should normally be the name of the program that will be run from the command line.

Extending a tool is just a matter of adding a register call. For example to add gcc support for the Sun OS would look like:

```

gcc.Register(

    hosts=[SystemPlatform('sunos','any')],

    targets=[SystemPlatform('sunos','any')],

    info=[

        GnuInfo(

            install_scanner=[

                PathFinder(['/usr/sfw/bin'])

            ],

            opt_dirs=[

                '/opt/'

            ],

            script=None,

            subst_vars={},

            shell_vars={'PATH':'${GCC.INSTALL_ROOT}'},

            test_file='gcc'

        )

    ]

)

```

Notice that in the case of gcc, there is a derived ToolInfo object called GnuInfo. This object was defined to make it easier to scan for different versions that are often installed side by side for most GNU tools. This object has a built in logic that requires a little more work than just defining an external Scanner as one might see for the Intel compiler. In this case it will find gcc and other common forms such as gcc-3.4. It also has the logic to get the version information for most GNU based tools via parsing the output of running the tool with the -v option.

In the case of the Intel tools, Scanners are used, because of a large possible range of versions that can be found. To help with defining the version range a new derived class of ToolInfo is defined. It is very simple, and uses the Parts VersionRange object:

```
class IntelcInfo(ToolInfo):  
  
    def __init__(self, version, install_scanner, script, subst_vars,  
shell_vars, test_file):  
  
        ToolInfo.__init__(self, version, install_scanner, script,  
subst_vars, shell_vars, test_file)  
  
        self.version=version_range(version)  
  
  
    def version_set(self):  
  
        return self.version
```

When used with a scanner we can define this:

```

# 64-bit 10.x

Intelc.Register(

    hosts=[SystemPlatform('win32','any')],

    targets=[SystemPlatform('win32','x86_64')],

    info=[

        IntelcInfo(

            version='10.*',

            install_scanner=regscanner.reg_scanner(

                [r'Software\Wow6432Node\Intel\Compilers\C++',

                 r'Software\Intel\Compilers\C++'],

                common.intel_10,

                'EM64T',

                'ICPP_COMPILER10','10'),

            script=ScriptFinder( '${INTELC.INSTALL_ROOT}/bin/ICLVars.bat'),

            subst_vars={

            },

            shell_vars={

                'PATH': '${INTELC.INSTALL_ROOT}/bin/',

                'INCLUDE': '${INTELC.INSTALL_ROOT}/include/',

                'LIB': '${INTELC.INSTALL_ROOT}/lib/'

            },

            test_file='icl.exe'

        )

    ]

)

```

Here the Intelc tool defined a registry scanner under IntelCommon. All scanners need to define scan() functions which will return a dictionary of version keys and install root path values and a resolve function that takes a

version string and returns the install root to the version if it is found, else None. The install root scanners should also cache results as this will greatly reduce startup time of SCons. Another example of a scanner is with Intelc tool as well. The windows side uses a `reg_scanner` object while posix systems uses a `file_scanner` objects. For example:

```

Intelc.Register(

    hosts=[SystemPlatform('posix','any')],

    targets=[SystemPlatform('posix','x86')],

    info=[

        IntelcInfo(

            version='11.*',

            install_scanner=filescanner.file_scanner11(

                '/opt/intel/Compiler',

                common.intel_11_outer,

                common.intel_11_inner,

                'ia32',

                'ICPP_COMPILER11'),

            script=ScriptFinder(
'${INTELC.INSTALL_ROOT}/bin/ia32/iclvars_ia32.bat'),

            subst_vars={

            },

            shell_vars={

                'PATH': '${INTELC.INSTALL_ROOT}/bin/ia32/',
                'INCLUDE': '${INTELC.INSTALL_ROOT}/include/',

                'LIB': '${INTELC.INSTALL_ROOT}/lib/ia32/'

            },

            test_file='icc'

        )

    ]

)

```

Scanner object is generally useful over using the finder objects when the look up logic requires finding lots of side by side versions of the tools in a layout location. The Scanner object centralizes the lookup logic while reusing most of the original code in ToolInfo object. Sub classing ToolInfo objects is generally useful when the look up logic

requires a lot of differently named executable, such as gcc, gcc-3, gcc-3.4, and different locations to look into, such as looking in /opt/gcc-3.4.6/bin.

